UNIVERSITY OF LONDON
INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects

CM3070 PROJECT
FINAL PROJECT REPORT

<Project title>

Author: Ananadakumar Kavyakrithika
Student Number : 200552932
Date of Submission: 25 March 2023
Supervisor : Mr Don Wee

Contents

CHAPTER 1: INTRODUCTION

Project Template 1: Deep Learning on a Public Dataset.

Project Concept: To predict whether high social media usage affects mental health in youths and young adults.

Project Concept

Mental health is a global pressing issue that many countries are trying to address. According to the 2022 World Mental Health Report by the World Health Organisation (WHO), One in eight people live with mental health disorders (*TEAM, World Mental Health Report: Transforming mental health for all 2022*). Suicide seems to be the most common cause of death among teenagers and young adolescents. What then affects the mental health of young adolescents and teenagers? Well, social media plays a major role in this. This is proven in the 2022 Global Gen Z Survey conducted by McKinsey Health Institute (MHI), where one in four youths responded that their mental health declined over the last three years(*Coe et al., Gen Z mental health: The impact of tech and Social Media 2023*). They were also the group that spent the most amount of time on social media. It is unlikely for youths and young adults to stay away from using social media platforms in this digital world where these platforms and their interactions are in high prevalence. Social media has become part of their lives and also acts

as a distressing activity. However, social media usage is proven to be the main cause of mental health disorders among youths according to the 2023 report by the U.S Surgeon General. Hence, our project aims to analyze and predict whether high social media usage affects mental health in youths and young adults to provide more concrete evidence and back up this claim.

Motivations

Tech companies are constantly making changes to their social media platforms to provide a safer and stress-free environment for youths and young adults to continue to use these platforms more healthily. In 2022, Instagram concealed the number of likes on posts to allow users to feel less pressurized and competitive, in an attempt to help improve the mental health of users. However, this attempt seems to create a lesser impact as users can continue to post comments and are still able to compare themselves with others. In another attempt, Social Media Platforms such as TikTok and Snapchat have a minimum age requirement for users to be at least thirteen years old. However, according to the 2023 report by the U.S. Surgeon General, about 40% of children aged between eight and twelve years old use these platforms. Despite age restrictions and changing platform features, Social media still causes mental health issues in teenagers and young adults as these platforms are built to be addictive and pleasurable. Hence, we need to analyze the time spent by youths and young adults on these platforms and prove that high social media usage affects their mental health.

Key questions to take into consideration:

- Do other factors such as low self-esteem and sleep deprivation affect the mental health of our target audience?
- Are the current available models effective in predicting the claim?
- What are the impacts of the type of platforms on the outcome?

Deliverables

Firstly, a baseline model has to be established after comparing various classification models. The baseline model will set a score for minimum performance and when our model performs better than the score set, then our model is proven to be efficient. The baseline model is also effective in evaluating other models that might aid us in analyzing the features of the dataset.

Secondly, a machine-learning model has to be built. The model should be able to accurately analyze and classify youths who are at risk of developing mental health issues based on the number of hours they spend using social media platforms. We will inspect how other features affect the intended outcome during the process of building and training the model. This might include the relationship status and even the type of platforms they use. A well-trained model can consider these features and provide an accurate prediction of the results.

Lastly, the model will be evaluated using evaluation metrics. Evaluation metrics will enable us to analyze the effectiveness of the model in predicting the intended outcome. They also allow us to identify the flaws in our model and help us to improve the model by addressing these issues.

CHAPTER 2: LITERATURE REVIEW

A hybrid deep learning approach for depression prediction from user tweets using feature-rich CNN and bi-directional LSTM

This paper aims to identify depression in Twitter data by proposing a model that is a combination of the Convolution Neural Network(CNN) and a bi-directional Long Short-Term Memory(biLSTM) which achieved an accuracy score of 94.28%(*Kour et al., An hybrid deep learning approach for depression prediction from user tweets using feature-rich CNN and bi-directional LSTM 2022*). The Recurrent Neural Network(RNN) and CNN models are used as baseline models for evaluation. A confusion matrix and AUC curves are also used to evaluate the model.

The biLSTM model is efficient in tackling the vanishing and exploding gradient problems faced by the basic RNN network as it analyses the meaning of a particular word in both directions in a sequence. When the CNN model is combined with the biLSTM model, outstanding results are achieved as the combined model can make use of the bidirectional function of the biLSTM and the feature extraction ability of the CNN model to produce more accurate results. When compared to a classic RNN model, the hybrid model can perform more efficiently as it tackles gradient-related problems to achieve a more precise intended outcome. The biLSTM model can also process longer sequences and CNN uses the backpropagation algorithm to understand and extract important features from the input. Hence, a combination of different deep learning models can interpret datasets more in-depth and more effectively and produce results with a higher accuracy score compared to basic deep learning models such as CNN and RNN. Thus, in this project, a combination of models will be used to evaluate and classify different features in the dataset chosen to build a hybrid model that can detect mental health issues among youths and young adults.

Features such as the number of hours spent on social media, the type of platforms the users use, sleep levels, and depression levels will be taken into consideration when building the hybrid model.

Big Data Analytics on Social Networks for real-time depression Detection

The work proves that Random Forest is the most effective machine learning technique in analyzing tweets with demographic features and sentiment analysis for depression in real-time(*Angskun et al., Big Data Analytics on Social Networks for real-time Depression Detection - Journal of Big Data 2022*). The work compares five machine learning techniques, namely Support Vector Machine(SVM), Decision Tree, Naive Bayes, Random Forest, and Deep Learning. The Deep learning technique uses the ReLU activation function with deep feedforward networks that contain a hundred hidden layers(*cite*). The SVM is used as a baseline model for comparison and the models will be evaluated using evaluation metrics such as F-Measure, Accuracy, Precision, and Recall.

The Random Forest and Decision Tree techniques had a higher accuracy score than the baseline model. Still, Random Forest seems to perform better as it is an upgraded model of Decision Tree. The Random Forest Technique provided more precise results as it integrates multiple decision tree outputs to produce a single outcome. This makes it an efficient model that can analyze live Twitter tweets for signs of depression. The Random Forest Technique is also easier to use and adapts easily to solve classification and regression problems. Evaluating live tweets and categorizing them accordingly requires the model to be able to examine huge amounts of data at a faster rate, Random Forest seems to be the most appropriate model as it can handle large amounts of data sets skillfully and has high precision levels when predicting the intended outcomes.

However, the downfall of using the Random Forest technique is that more computational resources are needed when using it. It is also an extremely complex model to comprehend due to the vast number of decision trees created to evaluate a large dataset. Random Forest is also set to be slower than the decision tree algorithm and requires a longer time to deduce the results. Hence, the Random Forest technique is not suitable to use in this project as my model needs to be quicker in evaluating the large amounts of data on the time spent by the user on social media platforms, and the type of platforms the users use, and other factors such as lack of sleep of the user causes the development of mental health issues in youths and young adults.

Multimodal depression detection on Instagram considering the time interval of posts

This paper uses a multimodal system to inspect images, text, and behavioral features to predict the depression score of each Instagram post. For analyzing images, the Convolutional Neural Network(CNN) model is used. For the text datatype, a compilation of Recurrent Neural Network(RNN) models such as Long short-term memory (LSTM), Bi-Directional Long short-term (BiLSTM), Gated Recurrent Unit(GRU), and bi-directional Gated Recurrent Unit(BiGRU) are used to build a text classification model. For behavioral evaluation, the Random Forest Classifier is used.

The use of three different models to examine three different types of data is proven to be effective as the model has an F1 score of 0.835. Using the CNN model for image classification is a good choice as it can independently extract features and patterns at a huge scale without having to use feature engineering which makes it an efficient model for processing images. The RNN model is suitable for examining the text data in Instagram posts as there are time intervals present and it is important to consider this aspect when extracting the key features from the text data. RNN models have a memory of history which allows them to capture the background of the input text and evaluate it accordingly(*cite*). A combination of RNN models will allow us to eliminate the vanishing and exploding gradients that basic RNN models experience. The behavioral features are extracted through the Random Forest classifier as it overcomes overfitting and can identify features that are of more importance than others.
Hence, using a combination of different deep learning models to examine the different types of data separately and combining the aggression scores to predict and classify users with depression is an efficient method that can be used in this project. The dataset for this project also contains different datatypes such as numerical and text data which can be evaluated separately using Feed-Forward neural networks for numerical data and RNN model for text data. Then, they can be combined using aggression scores which can then be used to predict and classify youths and young adults who are at risk of developing mental health issues.

Stress detection using natural language processing and machine learning over social interactions

This paper aims to examine tweets using Machine learning algorithms such as Random Forest, Decision Tree, and Logistic Regression for sentiment analysis, and the deep learning model Bidirectional Encoder Representations from Transformers(BERT) is used for sentiment classification. The BERT model was able to efficiently identify emotions with an accuracy rate of 94%. Among the three machine learning models used, Random Forest managed to produce the most accurate results with an accuracy of 97.78%(*Nijhawan et al., Stress detection using natural language processing and machine learning over social interactions - Journal of Big Data 2022*). This shows that two different models are used for analysis and classification to enhance the accuracy and precision of the intended outcome.

The BERT model is used for sentiment analysis where a transformer is used to examine the input text data in a bidirectional method which allows it to learn the meaning of a particular word based on its accompanying neighbors. This allows the model to produce a more accurate result compared to other natural language processing models. A Random Forest technique is proven to be the most efficient method in classifying tweets according to their stress levels. As evaluated in the previous sections, Random Forest is effective in providing a more precise result by analyzing large amounts of data but is slower and requires a large amount of resources for computation. In conclusion, we can interpret that it is decisive to use two different models for analysis and classification to achieve results with higher accuracy. However, the dataset I will be using in this project makes use of numerical data and scales mostly apart from the relationship status and the type of social media platforms that are used by users. Hence, a separate model is not required for sentiment analysis; only a combination of classification models can efficiently detect mental health issues among youths and young adults.

CHAPTER 3:PROJECT DESIGN

Domain and Users

The project's scope is focused on how long hours spent by the young generation on social media platforms cause them to develop mental health issues. Popular social media platforms such as TikTok and Instagram do not restrict the time spent by users on their platforms. Since these platforms are also made to be interactive and engaging, users tend to lose track of time and spend more hours browsing through the content than they were intended to do. For example, constant late-night scrolling causes a lack of sleep among users which negatively affects their physical well-being over time. Other negative effects include a decrease in attention span, neglect of priorities, and quick mood changes. Hence, it is important to control and create awareness among users to restrict their time spent on these platforms. The target audience of the project includes youths and young adults who make up the largest group that spends the most time on these platforms. They are also more vulnerable and prone to developing mental health issues soon.

Design Choices

The dataset I have chosen is obtained from the Kaggle website and is titled “Social Media and Mental Health”. The dataset contains messy textual data and the numerical data have to be normalized and converted into tensors before being fed through the deep learning model. Hence, it is important to preprocess and prepare the data in the appropriate format that is suitable to be used as input for the deep learning model. Feature Engineering will also be performed on the dataset with categorical features and convert them into numbers which are easier for the model to comprehend.

A hybrid model of a Bi-directional Long Short Term Memory Model (bi-LSTM) and a Convolutional Neural Network(CNN) model will be built to analyze and predict

the numerical and textual data types present in the dataset. Then an aggression method will be used to extract the aggression scores of the two models which will then be used to classify and predict users who are more likely to develop mental health issues. The bi-LSTM model can tackle the gradient-related problems faced by the classic Recurrent Neural Network(RNN) model and increase the efficiency and accuracy of the model in general. Since CNN models are great at identifying patterns and use feature engineering by themselves, it will be quicker to identify features that affect the time spent on social media platforms and convert them into numerical features to analyze and predict users who are at risk of developing mental health issues. The users will be classified into three categories, namely, Low risk, Moderate risk, and High risk according to the number of hours they spend on social media platforms.

Overall Structure

The project will be built on the Jupyter Notebook on my computer. Important libraries will be imported to run the program and build the deep learning model with ease. The dataset will undergo preprocessing steps and will be cleaned before being analyzed and visualized. Feature Engineering will be performed to convert categorical features that affect the time spent on social media platforms into numerical data types for easier processing. The dataset will then be regularized and converted to the appropriate data structure for building the baseline model. Then, the model is built step by step from simple to advanced deep learning models. A common-sense baseline model will be created for comparison before a simple machine-learning model with two dense layers is built. Then a basic Recurrent Neural Network(RNN) model is built using LSTM layers. This model is then further built on and a Stacked RNN model is produced. The Stacked RNN is then developed into an advanced model of the Bi-LSTM model. A CNN model is then developed and combined before the aggression method is used to classify and predict. This can be seen from the workflow and

system architectures displayed below in Figures 1.1 and 1.2. The performance of the model is then evaluated followed by the tuning of hyperparameters, and predictions are performed after training the model. The models are developed using the Python language and the baselines are built through either simple algorithms or pipelines to achieve results with high statistical power.

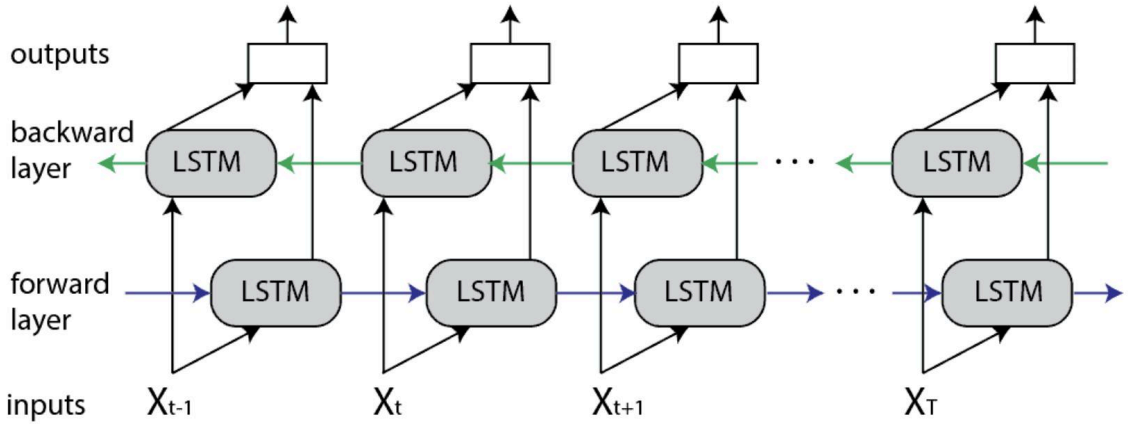


Figure 1.1 The bi-LSTM Architecture

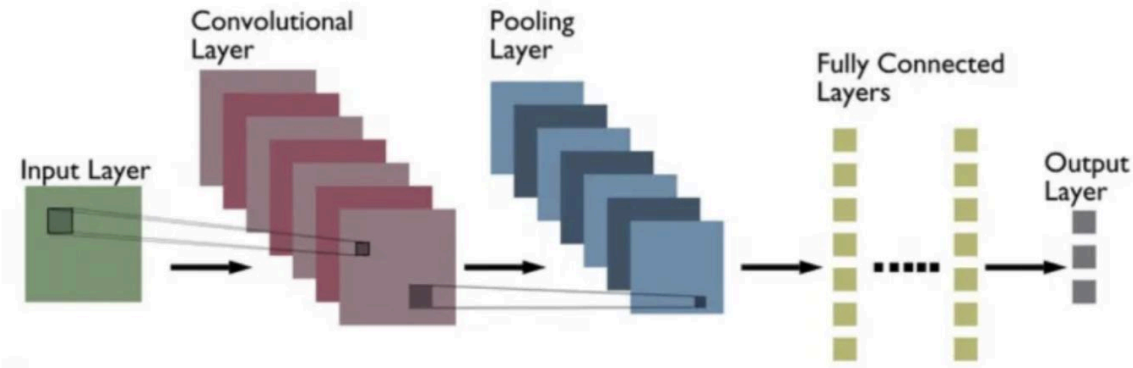


Figure 1.2 The CNN Architecture

Software and Methods Used

Supervised Learning

Supervised learning is used in this project as it makes use of labeled data to analyze and classify them accurately and can make precise predictions on

unseen data. This technique is used to build and train the hybrid model of bi-LSTM and CNN models where they can classify and predict youths and young adults who are at risk of developing mental health issues. The bi-LSTM model is effective in grasping long-term dependencies and can capture important features from input data effectively as it can analyze both current and past inputs and make accurate predictions. The CNN model can be used to interpret the patterns present in the features, making it an efficient model for classifying and predicting the intended outcome.

Universal Workflow of Machine Learning

The universal workflow of machine learning acts as a framework that can be used to solve the machine learning problem that we have right now. There are seven steps in this framework and they are firstly, defining the problem and assembling a dataset. Secondly, Choosing a measure of success. Thirdly, Deciding an evaluation protocol. Fourthly, preparing the data. Next, developing a model that does better than a baseline. Then, scaling up where an overfitting model is developed. Lastly, regularize the model and tune its hyperparameters(*cite*). This workflow is displayed in Figure 1.3 shown below. The project follows this workflow closely for the development and evaluation of the model.

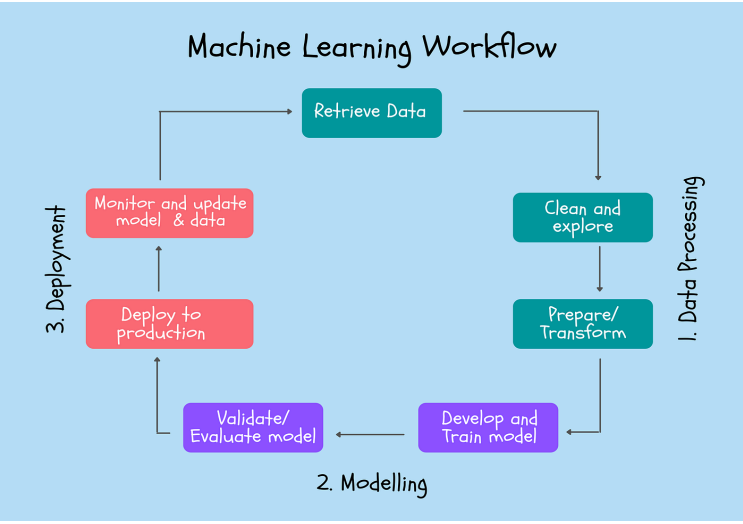


Figure 1.3 The universal workflow of machine learning

Language Used

Python will be used as the main language to develop the baseline models and the hybrid model due to its availability of an extensive library and packages. It is a user-friendly platform that can deliver projects of great quality.

Work Plan

The work plan can be seen in A1, A2, and A3 in Appendices.

Evaluation and Testing

The model will also be compared with the baseline model to ensure that it performs better than the benchmark and can predict the outcome accurately. As seen from the Literature Review, deep learning models such as CNN and RNN models are effective in analyzing, classifying, and predicting satisfactory results. Hence, the model will be compared with these existing deep learning models and how well it performs in identifying and classifying youths and young adults who are at risk of developing mental health issues. An evaluation protocol such as Hold-Out is also used to measure the progress of the model.

The Evaluation metrics that are going to be used are Accuracy, F1 Score, and AUC-ROC(Area under the ROC curve) scores. Accuracy is used as an evaluation metric when training the deep learning models and the accuracy score is plotted.Since the data is imbalanced, it is suitable to use the F1 score as it considers imbalance when evaluating the model. Since the F1 score is an average of both Precision and Recall, it is more effective to use the F1 score to evaluate the model. The AUC-ROC scores are also used as an evaluation metric as they provide an accurate measure of the model performance and are commonly used for classification problems. Testing is then performed on the final hybrid model once its accuracy is calculated and evaluated to ensure that it can precisely classify the youths and young adults into their respective risk categories and performs better than the baseline and existing deep learning models.

CHAPTER 4:IMPLEMENTATION

Define the problem and assemble a dataset

The problem can be defined as a multiclass classification where the time spent by youths and young adults is analyzed and classified into the three different risk categories. One hot encoding is performed on the gender and the risk categories where the categorical features are converted to numerical values for easier processing as input data that can be fed into the deep learning network. This can be seen from Figures A32 and A33 in the Appendices.

Choosing a measure of success

The evaluation of the model is performed through the metrics defined in the Project Design under the Evaluation and Testing section. The Hold Out is used as an evaluation protocol as it is a simple and popular evaluation protocol that is commonly used in classification problems.

Data Preparation

Firstly, the data has to be prepared and cleaned. The columns for all the features are renamed for easier processing. Then missing values in the dataset are handled using the dropna() function where rows with missing values are dropped. Unnecessary columns such as the sleep trouble scale are removed. Since the target users are youths and young adults, salaried workers, and retired user values are also removed using the drop function. The time spent daily on social media platforms feature is converted from textual data to numerical data. Risk categories are also assigned to the number of hours spent on social media. The features of social media platforms and affiliated organizations are converted to First Normal Form(INF) to eliminate data values that are repeating. Unique values in the Gender column are combined as one column labeled as Others. The cleaned dataset is then saved locally. This can be seen from Figures A6 to A17.

Data Analysis and Visualization

In the Data Analysis section, the data is first checked for balance. The dataset seems to be imbalanced and hence F1 score and AUC-ROC are used as evaluation metrics. The central tendency of the features' time spent daily on social media platforms and Age is measured through mean, median, and mode calculations. The spread of the features is also measured where the Range, Standard Deviation, Variance, and Quartlies are calculated. This shows us the wide range of values the two features mentioned above have and how they can be used to classify and predict the youths and young adults at risk of developing mental health issues. Then, in the Data Visualization section, the data is

visualized based on four key questions. They are firstly, which age group spends the most time on social media? Is it the teenagers are young adults?. Secondly, Which are the most popular social media platforms among our target audience?. Thirdly, Do affiliated organizations and Occupation status affect the time spent on social media platforms daily?. Lastly, Does the status relationship and Gender affect the time spent on social media platforms?. These key questions help us to visualize the relationship between these features and how they affect the time spent on social media platforms daily. If there is a positive relationship between these features mentioned above and the time spent on social media platforms daily, then these will be taken into account when developing the deep learning model. A confusion matrix is also used to calculate the correlation between the features present in the dataset. This can be seen from Figures A18 to A30.

Feature Engineering

Feature Engineering techniques such as Normalization and One-hot Encoding are used to standardize and convert data values into the appropriate format to be used as inputs for the deep learning models. The time spent daily on social media platforms is normalized to standardize the data values within the feature to avoid bias. The number of standard deviations from the mean is calculated where the mean is zero and the standard deviation is one(*Verma, How to normalize data using scikit-learn in Python 2023*). One Hot Encoding is performed on the risk categories variable where the text data is converted into integers. A 1 is represented for value in that category and a 0 if its not. This reduces the number of categories and makes it easier for processing. This can be seen from the Figures A31 to A35.

Baseline model

A common-sense baseline model is developed using a simple machine learning algorithm, which in this case is a Random Forest classifier. As inferred from the Literature Review above, the Random Forest classifier seems to be the best and produces the most accurate results for classification problems. Hence, it is used as the baseline model for this project. The model is then evaluated through the F1 score metric. An F1 score of 0.27 is obtained. This is a low accuracy score and the deep learning model has to perform better than this. This can be seen in Figure A37.

Simple Machine Learning Model

A simple fully connected model is developed where the data is first flattened before it is run through two dense layers. There is no sigmoid activation function present in the last layer and the accuracy metric is used to calculate the loss. The training and validation loss is then displayed through a graph. This can be seen in Figures A39 and A40 in Appendices. The validation loss is close to the baseline model and is unable to outperform the baseline model. This is because the baseline model has access to a large amount of valuable knowledge compared to the simple machine learning model.

Recurrent Neural Network

The Recurrent neural network is built using the LSTM layers and the RMS prop optimizer is used with the accuracy used as a loss metric. A total of 50 epochs is used to train the model with 100 epochs for every step. The training and validation loss are plotted. The model seems to perform the highest at epoch 11 before the model starts overfitting as seen in Figures A42 and A43. To tackle overfitting, the model is regularized through dropout and L1 and L2 regularization and evaluated again using the evaluation metrics. The model is now trained for more than twice the amount of epochs as it takes longer to regularize the model. The model does not seem to overfit before twenty epochs and there is an improvement in the evaluation scores. This can be seen in Figures A45 and A46 in the Appendices.

Stacked Recurrent Neural Network

To improve the network capacity of the model and its performance power, an additional layer of a recurrent neural network is added to the regularised RNN model. The validation and training accuracy and losses seem to have improved slightly as seen in Figures A48 and A49 in the Appendices.

Bi-Directional Long Short-term Memory Model

The advanced deep learning model, bi-LSTM is developed with a last-layer activation function of the sigmoid and a loss function titled ‘categorical_crossentropy’. The accuracy and validation scores of the model are calculated and plotted as seen in Figures A51 and A52. The model seems to perform slightly better than the baseline model as it has an F1 score of 0.29 and an AUC-ROC score of 0.7 as seen in Figures A53 and A54. This shows that the bi-LSTM model outperforms and can better analyze and classify teenagers and youths who are at risk of developing mental health issues more accurately than a simple machine learning classification model such as Random Forest which is commonly known for producing the most accurate results for classification problems.

CHAPTER 5:EVALUATION

The bi-LSTM model is tested using the test dataset and the F1 score, and the AUC-ROC score are all calculated. The bi-LSTM model has produced an accuracy score of 50% as seen in Figure A53 in Apendices. An F1 score of 0.29 and an AUC-ROC score of 0.70 as seen in the table shown in Figure 1.4 below and in Figures A54 and A55. The hybrid model was not developed due to time constraints and hence the evaluation and testing are only performed on the bi-LSTM model. This shows that the bi-LSTM model can analyze and classify youths and young adults to their respective risk categories accurately to only a small extent. The F1 score and the AUC-ROC score of the deep learning models, RNN and CNN are also calculated to compare and evaluate the model performance of the bi-LSTM model against other commonly used deep learning models. This can be seen in Figures A56 and A57. The RNN model has an F1 score of 0.28 and an AUC-ROC score of 0.72 while the CNN model has an F1 score of 0.24 and an AUC-ROC score of 0.50.

The bi-LSTM model seems to perform the best among the group of three as it has the highest F1 and AUC-ROC scores. The CNN model seems to perform the lowest among the group since the input data is in a numerical format and CNN models are more accurate in classifying images. The bi-LSTM seems to perform slightly better than the RNN model as it can handle gradient exploding and vanishing problems better than a simple RNN model. Since the bi-LSTM model can combine LSTM layers from both directions, it can produce a more significant output compared to other deep learning models. The disadvantage of the bi-LSTM model is that it is slow in analyzing and classifying data and requires a longer time to be trained. It is also able to only correctly classify the targeted users to a small extent of 50% and it has to be further trained with a larger dataset to improve its accuracy.

	Bi-LSTM	RNN	CNN
F1 Score	0.29	0.28	0.24
AUC-ROC score	0.70	0.72	0.50

Figure 1.4 Table showcasing the scores of the evaluation metrics.

CHAPTER 6:CONCLUSION

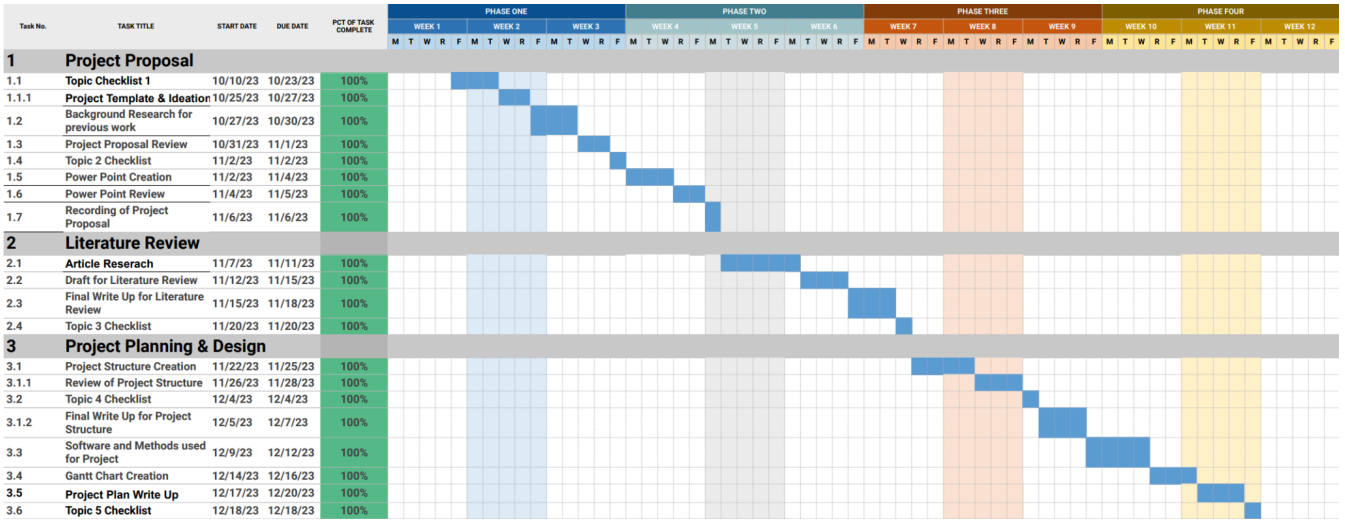
The project aimed to develop a deep learning model that can analyze and predict youths and young adults who are at risk of developing mental health issues. A common sense baseline model was built and the F1 score, and AUC-ROC scores are calculated to serve as a basis of comparison and to build a deep learning model that performs better than the baseline. Then, a simple machine learning model with two dense layers is built, and the validation and training losses are calculated and plotted as graphs. Accuracy is used as an evaluation metric in all models during training and validation. The simple machine learning model is then further improved upon and a RNN model is built. The validation and training losses are once again calculated and plotted as graphs. The RNN model seems to slightly perform better than the machine learning model. Then, the RNN model is regularized for overfitting using the dropout and L1 and L2 regularization techniques. The simple RNN model is further built on by adding a layer and a stacked RNN model is produced. The stacked RNN model performs better than the simple RNN model. The stacked RNN model is then developed further into a bi-LSTM model which is then tested using the test dataset and the model's performance is evaluated by comparing against other deep learning models such as RNN and CNN. The bi-LSTM model seems to perform the best among the three and is the most accurate in predicting and classifying youths and young adults who are at risk of developing mental health issues. However, due to time constraints, the initial plan of developing a hybrid model of bi-LSTM with CNN could not be implemented which can be used as part of future work for this project.

The project can be developed further in the future where more types of data such as text and images can be combined into one dataset and using the hybrid model of the bi-LSTM and CNN, we can analyze and predict youths and young adults who are at risk of developing mental health issues. Through tweets or Instagram posts of users, we can identify the type of posts posted and liked by the user and the time spent on the applications on average, we can identify trends and how likely the user is at risk of developing mental health conditions. To analyze how effective the bi-LSTM model is, the

model can be used to identify the signs of depression among youths and young adults by analyzing long textual data such as Reddit threads.

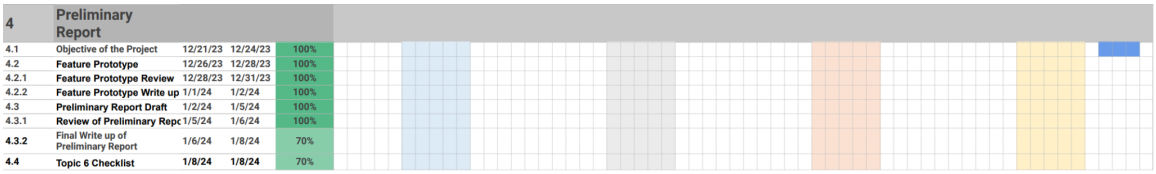
CHAPTER 7:APPENDICES

A1



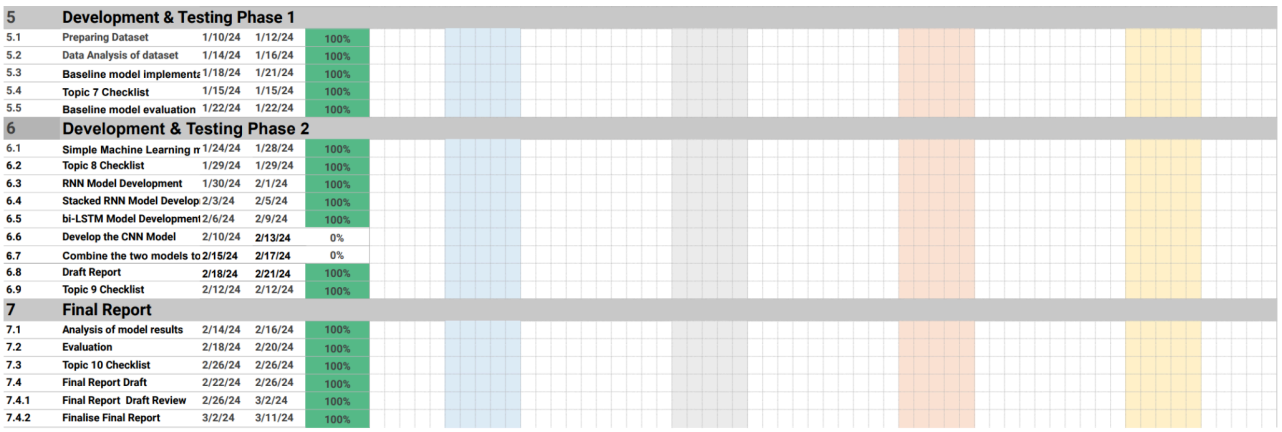
Task No.	TASK TITLE	START DATE	DUE DATE	PCT OF TASK COMPLETE
1	**Project Proposal**			
1.1	Topic Checklist 1	10/10/23	10/23/23	100%
1.1.1	Project Template & Ideation	10/25/23	10/27/23	100%
1.2	Background Research for previous work	10/27/23	10/30/23	100%
1.3	Project Proposal Review	10/31/23	11/1/23	100%
1.4	Topic 2 Checklist	11/2/23	11/2/23	100%
1.5	Power Point Creation	11/2/23	11/4/23	100%
1.6	Power Point Review	11/4/23	11/5/23	100%
1.7	Recording of Project Proposal	11/6/23	11/6/23	100%
2	**Literature Review**			
2.1	Article Reserach	11/7/23	11/11/23	100%
2.2	Draft for Literature Review	11/12/23	11/15/23	100%
2.3	Final Write Up for Literature Review	11/15/23	11/18/23	100%
2.4	Topic 3 Checklist	11/20/23	11/20/23	100%
3	**Project Planning & Design**			
3.1	Project Structure Creation	11/22/23	11/25/23	100%
3.1.1	Review of Project Structure	11/26/23	11/28/23	100%
3.2	Topic 4 Checklist	12/4/23	12/4/23	100%
3.1.2	Final Write Up for Project Structure	12/5/23	12/7/23	100%
3.3	Software and Methods used for Project	12/9/23	12/12/23	100%
3.4	Gantt Chart Creation	12/14/23	12/16/23	100%
3.5	Project Plan Write Up	12/17/23	12/20/23	100%
3.6	Topic 5 Checklist	12/18/23	12/18/23	100%

A2



4	Preliminary Report			
4.1	Objective of the Project	12/21/23	12/24/23	100%
4.2	Feature Prototype	12/26/23	12/28/23	100%
4.2.1	Feature Prototype Review	12/28/23	12/31/23	100%
4.2.2	Feature Prototype Write up	1/1/24	1/2/24	100%
4.3	Preliminary Report Draft	1/2/24	1/5/24	100%
4.3.1	Review of Preliminary Repo	1/5/24	1/6/24	100%
4.3.2	Final Write up of Preliminary Report	1/6/24	1/8/24	70%
4.4	Topic 6 Checklist	1/8/24	1/8/24	70%

A3



5	Development & Testing Phase 1			
5.1	Preparing Dataset	1/10/24	1/12/24	100%
5.2	Data Analysis of dataset	1/14/24	1/16/24	100%
5.3	Baseline model implementa	1/18/24	1/21/24	100%
5.4	Topic 7 Checklist	1/15/24	1/15/24	100%
5.5	Baseline model evaluation	1/22/24	1/22/24	100%
6	Development & Testing Phase 2			
6.1	Simple Machine Learning m	1/24/24	1/28/24	100%
6.2	Topic 8 Checklist	1/29/24	1/29/24	100%
6.3	RNN Model Development	1/30/24	2/1/24	100%
6.4	Stacked RNN Model Develop	2/3/24	2/5/24	100%
6.5	bi-LSTM Model Development	2/6/24	2/9/24	100%
6.6	Develop the CNN Model	2/10/24	2/13/24	0%
6.7	Combine the two models to	2/15/24	2/17/24	0%
6.8	Draft Report	2/18/24	2/21/24	100%
6.9	Topic 9 Checklist	2/12/24	2/12/24	100%
7	Final Report			
7.1	Analysis of model results	2/14/24	2/16/24	100%
7.2	Evaluation	2/18/24	2/20/24	100%
7.3	Topic 10 Checklist	2/26/24	2/26/24	100%
7.4	Final Report Draft	2/22/24	2/26/24	100%
7.4.1	Final Report Draft Review	2/26/24	3/2/24	100%
7.4.2	Finalise Final Report	3/2/24	3/11/24	100%

A4

In [1]:
```
!pip install fast_ml --quiet
```

In [2]:
```
#Import the libraries here
import pandas as pd
import numpy as np
from sklearn import preprocessing
from fast_ml.model_development import train_valid_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn import metrics
from keras.optimizers import RMSprop
import tensorflow as tf

from keras import models, regularizers, layers, optimizers, losses, metrics
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import np_utils, to_categorical
```

```
C:\Users\Kavya\Anaconda3\lib\site-packages\scipy\__init__.py:146: UserWarning: A NumPy version >=1.16.5 and <1.23.0 is requi
red for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}"
```

A5

In [62]:
```
#Load the dataset
Social_media_dataset = pd.read_csv('smmh.csv')
```

In [4]:
```
#Convert the dataset to dataframes
s_m_df = pd.DataFrame(Social_media_dataset)
#view the details of the dataframe
s_m_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 21 columns):
 #   Column
Non-Null Count  Dtype
---  ------
--------------  -----
 0   Timestamp
481 non-null    object
 1   1. What is your age?
481 non-null    float64
 2   2. Gender
481 non-null    object
 3   3. Relationship Status
481 non-null    object
 4   4. Occupation Status
481 non-null    object
 5   5. What type of organizations are you affiliated with?
451 non-null    object
```

A6

In [5]:
```
#The column names are renamed for easier processing of data
new_s_m_col_names = {
    'Timestamp': 'Timestamp',
    '1. What is your age?': 'Age',
    '2. Gender': 'Gender',
    '3. Relationship Status': 'status_relationship',
    '4. Occupation Status': 'status_occupation',
    '5. What type of organizations are you affiliated with?': 'affi_organisations',
    '6. Do you use social media?': 'use_social_media',
    '7. What social media platforms do you commonly use?': 's_m_platforms',
    '8. What is the average time you spend on social media every day?': 'time_spent_daily',
    '9. How often do you find yourself using Social media without a specific purpose?': 'freq_s_m_purpose',
    '10. How often do you get distracted by Social media when you are busy doing something?': 'freq_s_m_distracted',
    "11. Do you feel restless if you haven't used Social media in a while?": 'restlesness',
    '12. On a scale of 1 to 5, how easily distracted are you?': 'distract_scale',
    '13. On a scale of 1 to 5, how much are you bothered by worries?': 'worry_level_scale',
    '14. Do you find it difficult to concentrate on things?': 'concentration_difficulty',
    '15. On a scale of 1-5, how often do you compare yourself to other successful people through the use of social media?': '
    '16. Following the previous question, how do you feel about these comparisons, generally speaking?': 'comparison_feelings
    '17. How often do you look to seek validation from features of social media?': 'freq_seek_validation',
    '18. How often do you feel depressed or down?': 'freq_feel_depressed',
    '19. On a scale of 1 to 5, how frequently does your interest in daily activities fluctuate?': 'in_fluctuate_scale',
    '20. On a scale of 1 to 5, how often do you face issues regarding sleep?': 'sleep_trouble_scale',
}

new_s_m_df = s_m_df.rename(columns=new_s_m_col_names)
```

A7

In [6]:
```
print(new_s_m_df)
```

```
              Timestamp   Age  Gender status_relationship  \
0     4/18/2022 19:18:47  21.0    Male   In a relationship
1     4/18/2022 19:19:28  21.0  Female              Single
2     4/18/2022 19:25:59  21.0  Female              Single
3     4/18/2022 19:29:43  21.0  Female              Single
4     4/18/2022 19:33:31  21.0  Female              Single
..                   ...   ...     ...                 ...
476   5/21/2022 23:38:28  24.0    Male              Single
477    5/22/2022 0:01:05  26.0  Female             Married
478   5/22/2022 10:29:21  29.0  Female             Married
479   7/14/2022 19:33:47  21.0    Male              Single
480  11/12/2022 13:16:50  53.0    Male             Married

      status_occupation   affi_organisations use_social_media  \
0    University Student           University              Yes
1    University Student           University              Yes
2    University Student           University              Yes
3    University Student           University              Yes
4    University Student           University              Yes
..                  ...                  ...              ...
476    Salaried Worker  University, Private              Yes
477    Salaried Worker           University              Yes
478    Salaried Worker           University              Yes
479  University Student           University              Yes
480    Salaried Worker              Private              Yes

                                       s_m_platforms       time_spent_daily  \
0    Facebook, Twitter, Instagram, YouTube, Discord...  Between 2 and 3 hours
1    Facebook, Twitter, Instagram, YouTube, Discord...      More than 5 hours
2              Facebook, Instagram, YouTube, Pinterest  Between 3 and 4 hours
3                                Facebook, Instagram      More than 5 hours
4                         Facebook, Instagram, YouTube  Between 2 and 3 hours
..                                                 ...                   ...
476                       Facebook, Instagram, YouTube  Between 2 and 3 hours
477                                Facebook, YouTube  Between 1 and 2 hours
478                                Facebook, YouTube  Between 2 and 3 hours
479  Facebook, Twitter, Instagram, YouTube, Discord...  Between 2 and 3 hours
480                                Facebook, YouTube      Less than an Hour

     freq_s_m_purpose  ...  restlesness  distract_scale  worry_level_scale  \
0                   5  ...            2               5                  2
1                   4  ...            2               4                  5
2                   3  ...            1               2                  5
3                   4  ...            1               3                  5
4                   3  ...            4               4                  5
..                ...  ...          ...             ...                ...
476                 3  ...            3               4                  3
477                 2  ...            2               3                  4
478                 3  ...            4               3                  2
```

A8

In [7]:
```
#Check whether the dataset has missing values
new_s_m_df.isnull().values.any()
```

Out[7]:
```
True
```

In [8]:
```
#Identify if the dataset has any missing values in columns
new_s_m_df.isnull().all()
```

Out[8]:
```
Timestamp                   False
Age                         False
Gender                      False
status_relationship         False
status_occupation           False
affi_organisations          False
use_social_media            False
s_m_platforms               False
time_spent_daily            False
freq_s_m_purpose            False
freq_s_m_distracted         False
restlessness                False
distract_scale              False
worry_level_scale           False
concentration_difficulty    False
comparison_scale            False
comparison_feelings         False
freq_seek_validation        False
freq_feel_depressed         False
in_fluctuate_scale          False
sleep_trouble_scale         False
dtype: bool
```

A9

In [9]:
```
#Counting the NaN in each column
new_s_m_df.isnull().sum()
```

Out[9]:
```
Timestamp                  0
Age                        0
Gender                     0
status_relationship        0
status_occupation          0
affi_organisations        30
use_social_media           0
s_m_platforms              0
time_spent_daily           0
freq_s_m_purpose           0
freq_s_m_distracted        0
restlesness                0
distract_scale             0
worry_level_scale          0
concentration_difficulty   0
comparison_scale           0
comparison_feelings        0
freq_seek_validation       0
freq_feel_depressed        0
in_fluctuate_scale         0
sleep_trouble_scale        0
dtype: int64
```

In [10]:
```
#Identify if the dataset has any missing values in rows.
new_s_m_df.isnull().all(axis=1)
```

Out[10]:
```
0      False
1      False
2      False
3      False
4      False
       ...
476    False
477    False
478    False
479    False
480    False
Length: 481, dtype: bool
```

A10

In [11]:
```
#Counting the NaN in each row
new_s_m_df.isnull().sum(axis=1)
```

Out[11]:
```
0      0
1      0
2      0
3      0
4      0
      ..
476    0
477    0
478    0
479    0
480    0
Length: 481, dtype: int64
```

In [12]:
```
#Counting the total number of NaN values in columns
print(new_s_m_df.isnull().values.sum())
```

```
30
```

In [13]:
```
#Drop the NaN values in the affi_organisations column.
new_s_m_df.dropna(subset=['affi_organisations'], inplace=True)
new_s_m_df.head()
```

Out[13]:

	Timestamp	Age	Gender	status_relationship	status_occupation	affi_organisations	use_social_media	s_m_platforms	time_spent_daily	freq_s_m_purpo
0	4/18/2022 19:18:47	21.0	Male	In a relationship	University Student	University	Yes	Facebook, Twitter, Instagram, YouTube, Discord...	Between 2 and 3 hours	
1	4/18/2022 19:19:28	21.0	Female	Single	University Student	University	Yes	Facebook, Twitter, Instagram, YouTube, Discord...	More than 5 hours	
2	4/18/2022 19:25:59	21.0	Female	Single	University Student	University	Yes	Facebook, Instagram, YouTube, Pinterest	Between 3 and 4 hours	
3	4/18/2022 19:29:43	21.0	Female	Single	University Student	University	Yes	Facebook, Instagram	More than 5 hours	
4	4/18/2022 19:33:31	21.0	Female	Single	University Student	University	Yes	Facebook, Instagram, YouTube	Between 2 and 3 hours	

5 rows × 21 columns

A11

In [14]:
```
#Drop columns that are unnecessary
new_s_m_df = new_s_m_df.drop('sleep_trouble_scale', axis =1)
```

In [15]:
```
#Remove salaried workers as we only require data from youths and young adults
new_s_m_df = new_s_m_df.drop(new_s_m_df[new_s_m_df['status_occupation'] == 'Salaried Worker'].index)
print(new_s_m_df)
```

```
              Timestamp   Age  Gender status_relationship   status_occupation  \
0    4/18/2022 19:18:47  21.0    Male   In a relationship  University Student
1    4/18/2022 19:19:28  21.0  Female             Single  University Student
2    4/18/2022 19:25:59  21.0  Female             Single  University Student
3    4/18/2022 19:29:43  21.0  Female             Single  University Student
4    4/18/2022 19:33:31  21.0  Female             Single  University Student
..                  ...   ...     ...                ...                 ...
468  5/14/2022 17:33:37  18.0    Male             Single      School Student
469  5/14/2022 17:33:57  15.0    Male             Single      School Student
470   5/15/2022 2:48:02  20.0  Female             Single  University Student
471  5/16/2022 20:20:31  20.0  Female             Single  University Student
479  7/14/2022 19:33:47  21.0    Male             Single  University Student

    affi_organisations use_social_media  \
0           University              Yes
1           University              Yes
2           University              Yes
3           University              Yes
4           University              Yes
```

In [16]:
```
#Remove retires as we only require data from youths and young adults
new_s_m_df = new_s_m_df.drop(new_s_m_df[new_s_m_df['status_occupation'] == 'Retired'].index)
print(new_s_m_df)
```

```
              Timestamp   Age  Gender status_relationship   status_occupation  \
0    4/18/2022 19:18:47  21.0    Male   In a relationship  University Student
1    4/18/2022 19:19:28  21.0  Female             Single  University Student
2    4/18/2022 19:25:59  21.0  Female             Single  University Student
3    4/18/2022 19:29:43  21.0  Female             Single  University Student
4    4/18/2022 19:33:31  21.0  Female             Single  University Student
..                  ...   ...     ...                ...                 ...
468  5/14/2022 17:33:37  18.0    Male             Single      School Student
469  5/14/2022 17:33:57  15.0    Male             Single      School Student
470   5/15/2022 2:48:02  20.0  Female             Single  University Student
471  5/16/2022 20:20:31  20.0  Female             Single  University Student
479  7/14/2022 19:33:47  21.0    Male             Single  University Student

    affi_organisations use_social_media  \
0           University              Yes
1           University              Yes
2           University              Yes
3           University              Yes
```

A12

In [17]:
```
#Convert time spent daily to integers
new_s_m_df.loc[new_s_m_df['time_spent_daily'] == 'Less than an Hour', 'time_spent_daily'] = 0
new_s_m_df.loc[new_s_m_df['time_spent_daily'] == 'Between 1 and 2 hours', 'time_spent_daily'] = 1
new_s_m_df.loc[new_s_m_df['time_spent_daily'] == 'Between 2 and 3 hours', 'time_spent_daily'] = 2
new_s_m_df.loc[new_s_m_df['time_spent_daily'] == 'Between 3 and 4 hours', 'time_spent_daily'] = 3
new_s_m_df.loc[new_s_m_df['time_spent_daily'] == 'Between 4 and 5 hours', 'time_spent_daily'] = 4
new_s_m_df.loc[new_s_m_df['time_spent_daily'] == 'More than 5 hours', 'time_spent_daily'] = 5

new_s_m_df['time_spent_daily'] = new_s_m_df['time_spent_daily'].astype('int64')
print(new_s_m_df)
```

```
468        School, N/A              Yes
469             School              Yes
470            Company              Yes
471         University              Yes
479         University              Yes

                                     s_m_platforms  time_spent_daily  \
0    Facebook, Twitter, Instagram, YouTube, Discord...                 2
1    Facebook, Twitter, Instagram, YouTube, Discord...                 5
2              Facebook, Instagram, YouTube, Pinterest                 3
3                                  Facebook, Instagram                 5
4                         Facebook, Instagram, YouTube                 2
..                                                 ...               ...
468                                          Pinterest                 5
469               Instagram, YouTube, Discord, Reddit                  4
470  Instagram, YouTube, Snapchat, Discord, Reddit,...                 1
471                         Facebook, Instagram, YouTube                 5
479  Facebook, Twitter, Instagram, YouTube, Discord...                 2

     freq_s_m_purpose  freq_s_m_distracted  restlesness  distract_scale  \
```

A13

In [19]:
```
#Assign risk catogories to the number of hours spent daily on social media
Risk_catogeries = {'Safe': [0,1,2], 'Moderate': [3,4], 'High Risk': [5]}
new_s_m_df['Risk Category'] = new_s_m_df['time_spent_daily'].map({i: m for m, x in Risk_catogeries.items() for i in x})
```

In [21]:
```
print(new_s_m_df['Risk Category'])
```

```
0          Safe
1     High Risk
2      Moderate
3     High Risk
4          Safe
        ...
468   High Risk
469    Moderate
470        Safe
471   High Risk
479        Safe
Name: Risk Category, Length: 324, dtype: object
```

A14

In [22]:
```
#Convert s_m_platforms to 1NF
new_s_m_df['s_m_platforms']=new_s_m_df['s_m_platforms'].str.split(";")
new_s_m_df.explode(['s_m_platforms'])
```

Out[22]:

	Timestamp	Age	Gender	status_relationship	status_occupation	affi_organisations	use_social_media	s_m_platforms	time_spent_daily	freq_s_m_pur
0	4/18/2022 19:18:47	21.0	Male	In a relationship	University Student	University	Yes	Facebook, Twitter, Instagram, YouTube, Discord...	2	
1	4/18/2022 19:19:28	21.0	Female	Single	University Student	University	Yes	Facebook, Twitter, Instagram, YouTube, Discord...	5	
2	4/18/2022 19:25:59	21.0	Female	Single	University Student	University	Yes	Facebook, Instagram, YouTube, Pinterest	3	
3	4/18/2022 19:29:43	21.0	Female	Single	University Student	University	Yes	Facebook, Instagram	5	
4	4/18/2022 19:33:31	21.0	Female	Single	University Student	University	Yes	Facebook, Instagram, YouTube	2	
...	...	...	...	...	...	...	...	...	...	
468	5/14/2022 17:33:37	18.0	Male	Single	School Student	School, N/A	Yes	Pinterest	5	
469	5/14/2022 17:33:57	15.0	Male	Single	School Student	School	Yes	Instagram, YouTube, Discord, Reddit	4	
470	5/15/2022 2:48:02	20.0	Female	Single	University Student	Company	Yes	Instagram, YouTube, Snapchat, Discord, Reddit,...	1	
471	5/16/2022 20:20:31	20.0	Female	Single	University Student	University	Yes	Facebook, Instagram, YouTube	5	
479	7/14/2022 19:33:47	21.0	Male	Single	University Student	University	Yes	Facebook, Twitter, Instagram, YouTube, Discord...	2	

324 rows × 21 columns

A15

In [23]:
```
#Convert affi_organisations to 1NF
new_s_m_df['affi_organisations']=new_s_m_df['affi_organisations'].str.split(";")
new_s_m_df.explode(['affi_organisations'])
```

status_relationship	status_occupation	affi_organisations	use_social_media	s_m_platforms	time_spent_daily	freq_s_m_purpose	...	restlessness	distract_scale	worry_level_scal
In a relationship	University Student	University	Yes	[Facebook, Twitter, Instagram, YouTube, Discor...	2	5	...	2	5	
Single	University Student	University	Yes	[Facebook, Twitter, Instagram, YouTube, Discor...	5	4	...	2	4	
Single	University Student	University	Yes	[Facebook, Instagram, YouTube, Pinterest]	3	3	...	1	2	
Single	University Student	University	Yes	[Facebook, Instagram]	5	4	...	1	3	

A16

In [24]:
```
#Genders with unique values are combined together as 'Others' in a single category.
new_s_m_df.replace('Non-binary','Others', inplace=True)
new_s_m_df.replace('Nonbinary ','Others', inplace=True)
new_s_m_df.replace('NB','Others', inplace=True)
new_s_m_df.replace('unsure ','Others', inplace=True)
new_s_m_df.replace('Non binary ','Others', inplace=True)
new_s_m_df.replace('Trans','Others', inplace=True)
```

In [25]:
```
#print all the Genders
Gender = set(new_s_m_df['Gender'])
print(Gender)
```

```
{'Female', 'Male', 'Others'}
```

A17

In [26]:
```
#save the dataset locally
new_s_m_df.to_csv('new_social_media_dataset')
```

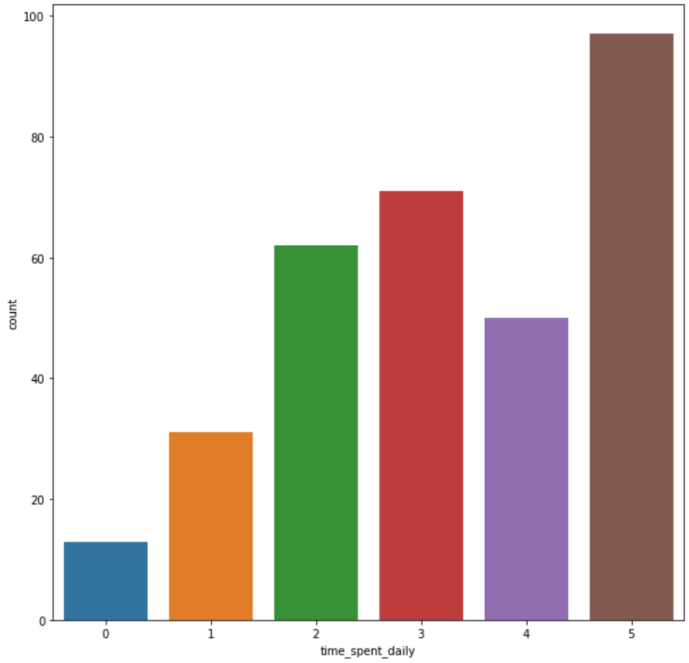
A18

Data Analysis

In [27]:
```
#Check whether the data is balanced?
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,10))
sns.countplot('time_spent_daily', data=new_s_m_df)
```

Out[27]: <AxesSubplot:xlabel='time_spent_daily', ylabel='count'>



A19

In [28]:
```
#Measures of Central Tendency for the features time_spent_daily and Age
#Mean, Median, Mode
#Calculating the mean time_spent_daily
new_social_media_dataset = pd.read_csv('new_social_media_dataset')
new_social_media_dataset_df = pd.DataFrame(data=new_social_media_dataset)
Mean_time_spent_daily = new_social_media_dataset['time_spent_daily'].mean()
print(Mean_time_spent_daily)
```

```
3.25
```

In [29]:
```
#Calculating the median time_spent_daily
Median_time_spent_daily = new_social_media_dataset['time_spent_daily'].median()
print(Median_time_spent_daily)
```

```
3.0
```

In [30]:
```
#Calculating the mode for time_spent_daily
Mode_time_spent_daily = new_social_media_dataset['time_spent_daily'].mode()
print(Mode_time_spent_daily)
```

```
0    5
dtype: int64
```

A20

In [31]:
```
#Calculating the mean Age
new_social_media_dataset = pd.read_csv('new_social_media_dataset')
new_social_media_dataset_df = pd.DataFrame(data=new_social_media_dataset)
Mean_Age = new_social_media_dataset['Age'].mean()
print(Mean_Age)
```

```
21.693518518518516
```

In [32]:
```
#Calculating the median Age
Median_Age = new_social_media_dataset['Age'].median()
print(Median_time_spent_daily)
```

```
3.0
```

In [33]:
```
#Calculating the mode for time_spent_daily
Mode_Age = new_social_media_dataset['Age'].mode()
print(Mode_Age)
```

```
0    21.0
dtype: float64
```

A21

In [34]:
```
#Measures of spread for features time_spent_daily and Age
#Range, Standard Deviation,Variance,Quartiles
#Range of spread of time_spent_daily
def Range_time_spent_daily(new_social_media_dataset_df):
    return  new_social_media_dataset_df.max() - new_social_media_dataset_df.min()
print(new_social_media_dataset_df[['time_spent_daily']].agg(Range_time_spent_daily))
```

```
time_spent_daily    5
dtype: int64
```

In [35]:
```
#Standard deviation of the time_spent_daily
print(new_social_media_dataset_df[['time_spent_daily']].agg(['std']))
```

```
     time_spent_daily
std          1.493795
```

In [36]:
```
#Variance of the time_spent_daily
Variance_time_spent_daily = new_social_media_dataset_df['time_spent_daily'].var()
print(Variance_time_spent_daily)
```

```
2.231424148606811
```

In [37]:
```
#Quartiles of the time_spent_daily
Quartiles_time_spent_daily = new_social_media_dataset_df['time_spent_daily'].quantile([0.25,0.5,0.75])
print(Quartiles_time_spent_daily)
```

```
0.25    2.0
0.50    3.0
0.75    5.0
Name: time_spent_daily, dtype: float64
```

A22

In [38]:
```
#Range of spread of Age
def Range_Age(new_social_media_dataset_df):
    return  new_social_media_dataset_df.max() - new_social_media_dataset_df.min()
print(new_social_media_dataset_df[['Age']].agg(Range_Age))
```

```
Age    78.0
dtype: float64
```

In [39]:
```
#Standard deviation of Age
print(new_social_media_dataset_df[['Age']].agg(['std']))
```

```
          Age
std  4.799915
```

In [40]:
```
#Variance of Age
Variance_Age = new_social_media_dataset_df['Age'].var()
print(Variance_Age)
```

```
23.039183866529072
```

In [41]:
```
#Quartiles of Age
Quartiles_Age = new_social_media_dataset_df['Age'].quantile([0.25,0.5,0.75])
print(Quartiles_Age)
```

```
0.25    20.0
0.50    21.0
0.75    23.0
Name: Age, dtype: float64
```

A23

In [42]:
```
#Type of Distribution of the data
#Discrete or continuous data for the two features
#Identifying the distribution for time_spent_daily
Time_spent_daily_distribution = new_social_media_dataset_df['time_spent_daily'].value_counts().describe()
print(Time_spent_daily_distribution)
```

```
count     6.000000
mean     54.000000
std      29.759032
min      13.000000
25%      35.750000
50%      56.000000
75%      68.750000
max      97.000000
Name: time_spent_daily, dtype: float64
```

In [43]:
```
#Identifying the distribution of Age
Age_distribution = new_social_media_dataset_df['Age'].value_counts().describe()
print(Age_distribution)
```

```
count    20.000000
mean     16.200000
std      23.831867
min       1.000000
25%       2.000000
50%       5.500000
75%      16.500000
max      79.000000
Name: Age, dtype: float64
```

A24

Data Visualisation

Which Age group spent the most time on social media platforms daily?

Which are the most popular social media platforms among youths and young adults?

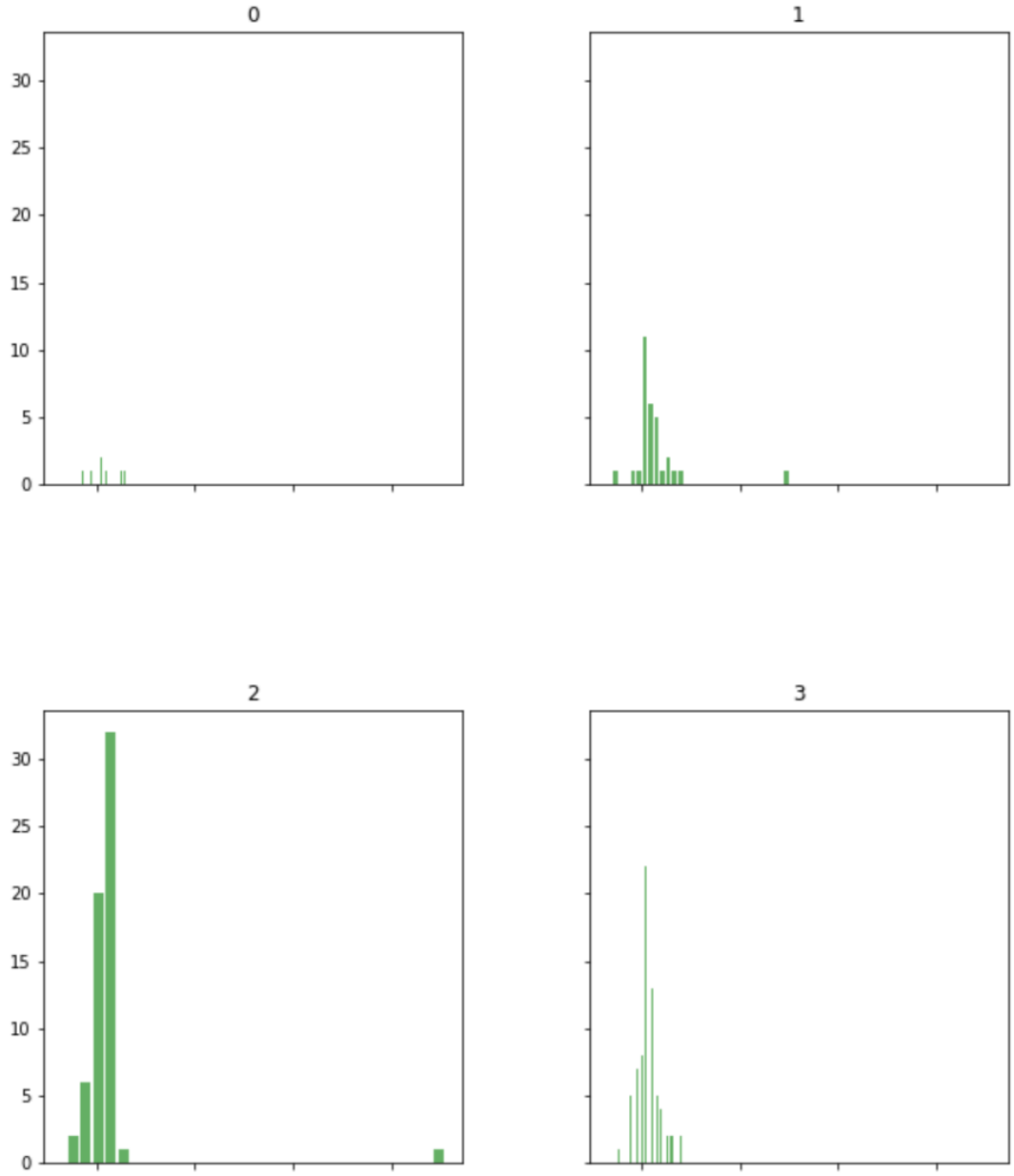
Does the affi_organisations and status_occupation affect the time spent on social media platforms daily?

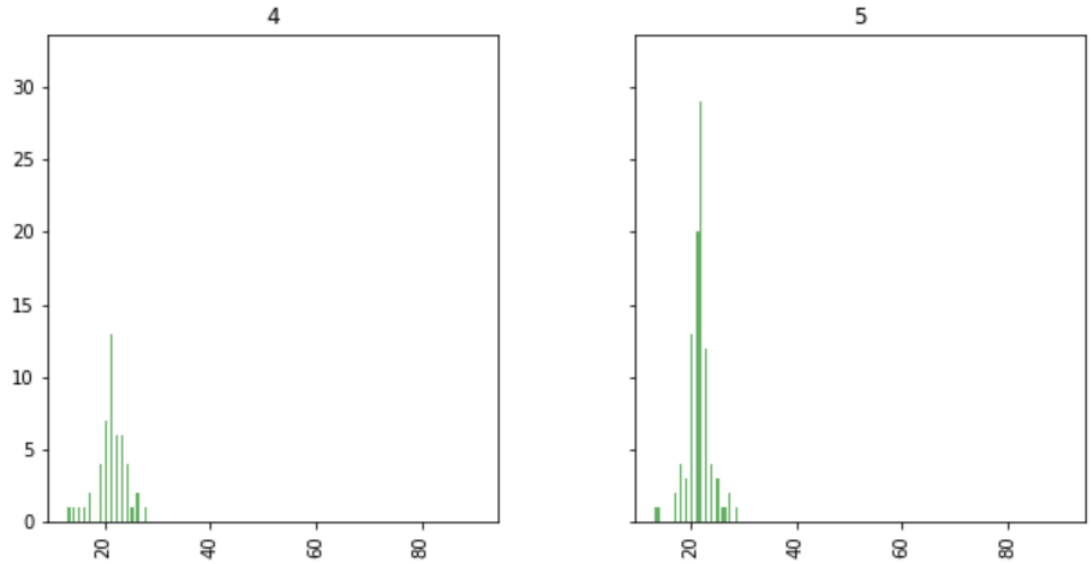
Do the status relationship and Gender affect the time spent on social media platforms?

In [44]:
```
#Which Age group spent the most time on Social Media platforms daily?

#Plot the histogram
Age_Group = new_social_media_dataset_df.hist(column='Age', by='time_spent_daily', bins=30,color='Green',
                            figsize=(10,20), alpha=0.6,
                            grid=False, rwidth=0.8,
                            sharex=True, sharey=True)

#The Histogram is displayed
plt.show()
```


0
1
2
3
0
5
10
15
20
25
30



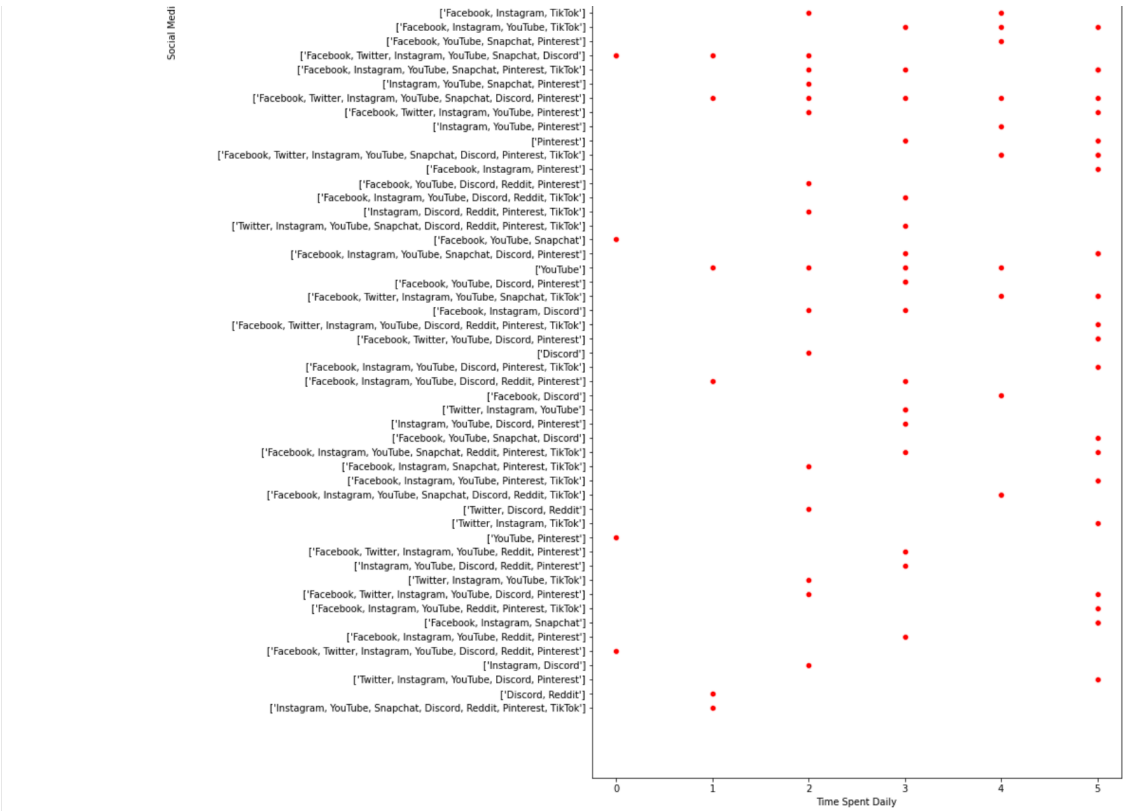
A25

In [45]:

```
#Which are the most popular social media platforms among youths and young adults?
#Plot the scatterplot
plt.figure(figsize=(10,30))
Social_Media_Platforms_Time_Spent = sns.scatterplot(data = new_social_media_dataset_df, x='time_spent_daily', y='s_m_platform
# The title and the x and y-axis labels are added
plt.xlabel('Time Spent Daily')
plt.ylabel('Social Media Platforms')
plt.title('Time Spent Daily vs Social Media Platforms')
#Display the scatterplot
print(Social_Media_Platforms_Time_Spent)
```

```
AxesSubplot(0.125,0.125;0.775x0.755)
```


Time Spent Daily vs Social Media Platforms
Social Media Platforms
['Facebook, Twitter, Instagram, YouTube, Discord, Reddit']
['Facebook, Instagram, YouTube, Pinterest']
['Facebook, Instagram']
['Facebook, Instagram, YouTube']
['Facebook, Twitter, Instagram, YouTube, Discord, Pinterest, TikTok']
['Facebook, Instagram, YouTube, Snapchat, TikTok']
['Reddit, Pinterest']
['Facebook, Instagram, YouTube, Discord']
['Facebook, Instagram, YouTube, Snapchat, Discord, Reddit, Pinterest, TikTok']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Discord, Reddit, Pinterest']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Reddit, Pinterest']
['Facebook, YouTube, Pinterest']
['Facebook, Instagram, YouTube, Reddit']
['Facebook, YouTube, Discord']
['Facebook, YouTube, Discord, Reddit']
['Facebook, Instagram, YouTube, Discord, Reddit']
['Facebook, Twitter, Instagram, YouTube, Discord, Reddit, TikTok']
['Facebook, Instagram, YouTube, Snapchat, Pinterest']
['Facebook, Instagram, YouTube, Snapchat, Discord, Pinterest, TikTok']
['Facebook, Instagram, YouTube, Snapchat, Discord']
['Facebook, Instagram, YouTube, Discord, Pinterest']
['Facebook, Twitter, Instagram, YouTube, Discord']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Discord, Reddit']
['Facebook, Instagram, YouTube, Snapchat']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Discord, Reddit, Pinterest, TikTok']
['Facebook']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Reddit, TikTok']
['Facebook, Instagram, YouTube, Snapchat, Discord, Reddit']
['Instagram, Reddit']
['YouTube, Reddit']
['YouTube, Discord, Reddit']
['Facebook, Instagram, Reddit, TikTok']
['Twitter, Instagram, YouTube, Reddit, TikTok']
['Reddit']
['YouTube, Snapchat, Discord, Reddit']
['Instagram, YouTube, Discord, Reddit']
['Twitter, Instagram, YouTube, Snapchat, Reddit, Pinterest, TikTok']
['Facebook, YouTube, Reddit']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Discord, TikTok']
['Facebook, Instagram, YouTube, Snapchat, Discord, Reddit, Pinterest']
['Instagram, YouTube, Snapchat, Discord, Reddit']
['Facebook, Twitter, Instagram, YouTube, Reddit']
['Facebook, YouTube']
['Facebook, Twitter, Instagram, YouTube, Snapchat']
['Twitter, Instagram, YouTube, Snapchat, Discord, Reddit, TikTok']
['Instagram, YouTube, Discord']
['Facebook, Instagram, YouTube, Snapchat, Discord, TikTok']
['Facebook, Twitter, Instagram, YouTube']
['Instagram']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Pinterest']
['Facebook, Instagram, TikTok']
['Facebook, Instagram, YouTube, TikTok']
['Facebook, YouTube, Snapchat, Pinterest']
['Facebook, Twitter, Instagram, YouTube, Snapchat, Discord']

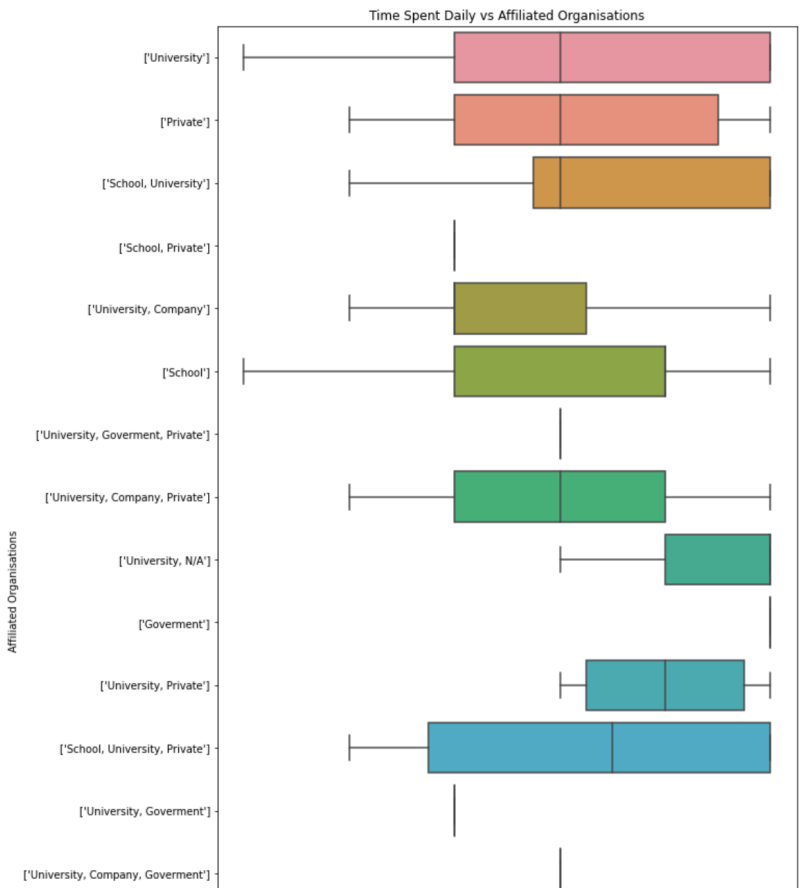


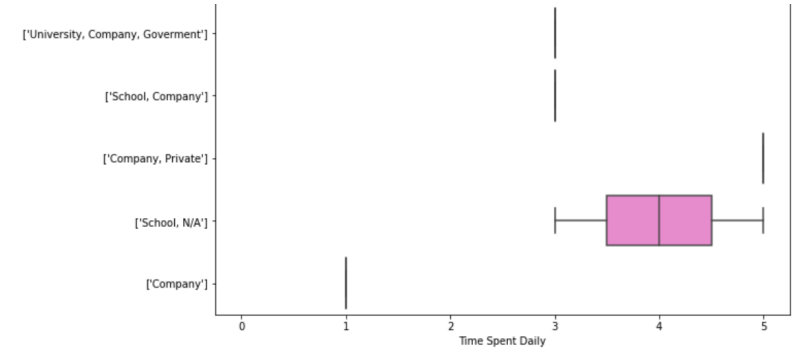
A26

In [46]:

```
#Does the affi_organisations affect the time spent on social media platforms daily?
#Plot the boxplot
plt.figure(figsize=(10,20))
Affi_organisations_Time_Spent = sns.boxplot(data = new_social_media_dataset_df, x='time_spent_daily', y='affi_organisations')
# The title and the x and y-axis labels are added
plt.xlabel('Time Spent Daily')
plt.ylabel('Affiliated Organisations')
plt.title('Time Spent Daily vs Affiliated Organisations')
#Display the boxplot
print(Affi_organisations_Time_Spent)
```

```
AxesSubplot(0.125,0.125;0.775x0.755)
```

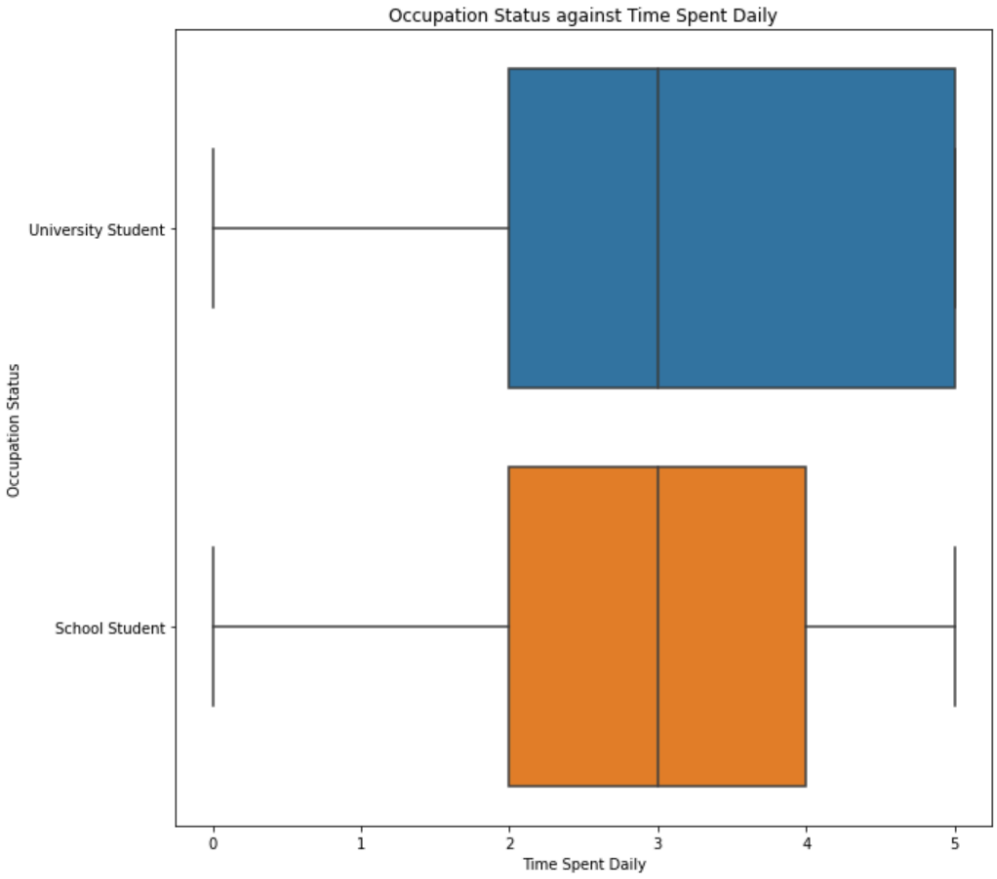

Time Spent Daily vs Affiliated Organisations
['University']
['Private']
['School, University']
['School, Private']
['University, Company']
['School']
['University, Goverment, Private']
['University, Company, Private']
['University, N/A']
['Goverment']
['University, Private']
['School, University, Private']
['University, Goverment']
['University, Company, Goverment']
Affiliated Organisations


['University, Company, Goverment']
['School, Company']
['Company, Private']
['School, N/A']
['Company']
0
1
2
3
4
5
Time Spent Daily

A27

In [47]:
```
#Does the status_occupation affect the time spent on social media platforms daily?
#Plot the boxplot
plt.figure(figsize=(10,10))
Status_Occupation_Time_Spent = sns.boxplot(data = new_social_media_dataset_df, x='time_spent_daily', y='status_occupation')
# The title and the x and y-axis labels are added
plt.xlabel('Time Spent Daily')
plt.ylabel('Occupation Status')
plt.title('Occupation Status against Time Spent Daily')
#Display the boxplot
print(Status_Occupation_Time_Spent)
```

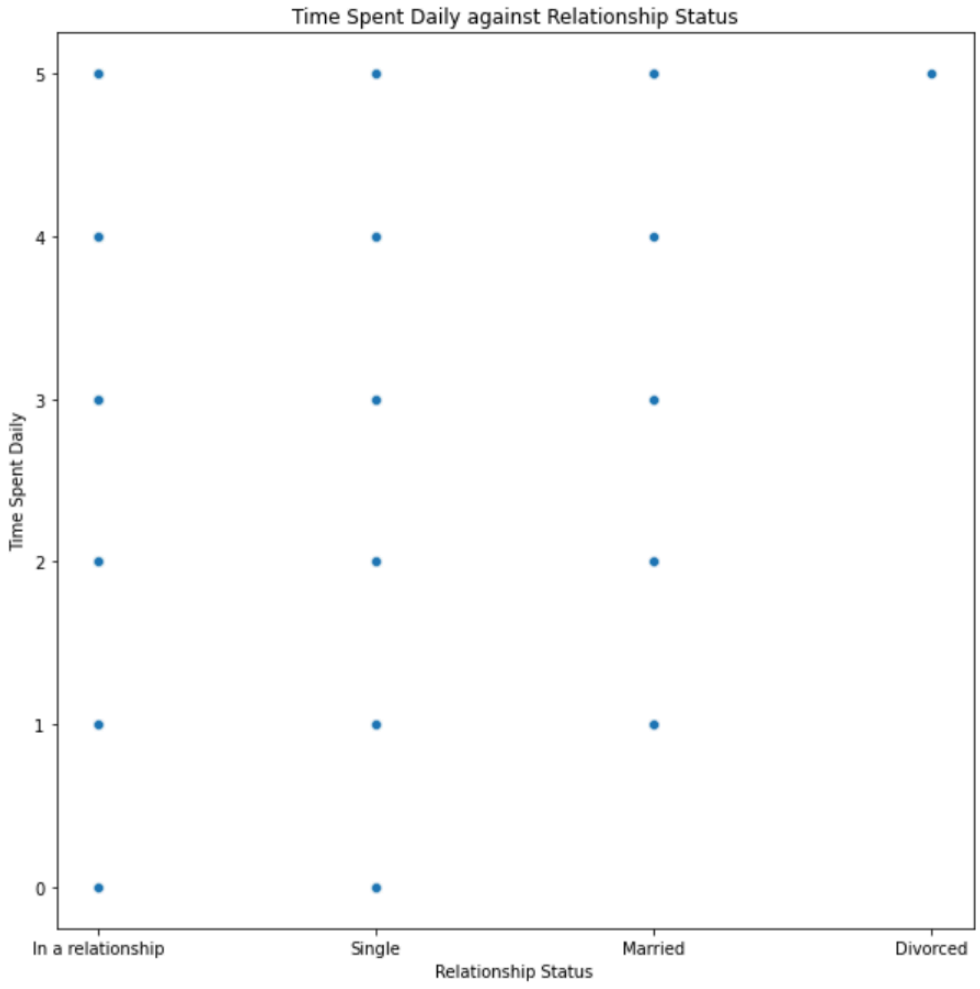
```
AxesSubplot(0.125,0.125;0.775x0.755)
```



A28

```
In [48]: #Does the status relationship affect the time spent on social media platforms?
         #Plot the scatterplot
         plt.figure(figsize=(10,10))
         Status_Relationship_Time_Spent = sns.scatterplot(data = new_social_media_dataset_df, x='status_relationship', y='time_spent_d
         # The title and the x and y-axis labels are added
         plt.xlabel('Relationship Status')
         plt.ylabel('Time Spent Daily')
         plt.title('Time Spent Daily against Relationship Status')
         #Display the scatterplot
         print(Status_Relationship_Time_Spent)
```

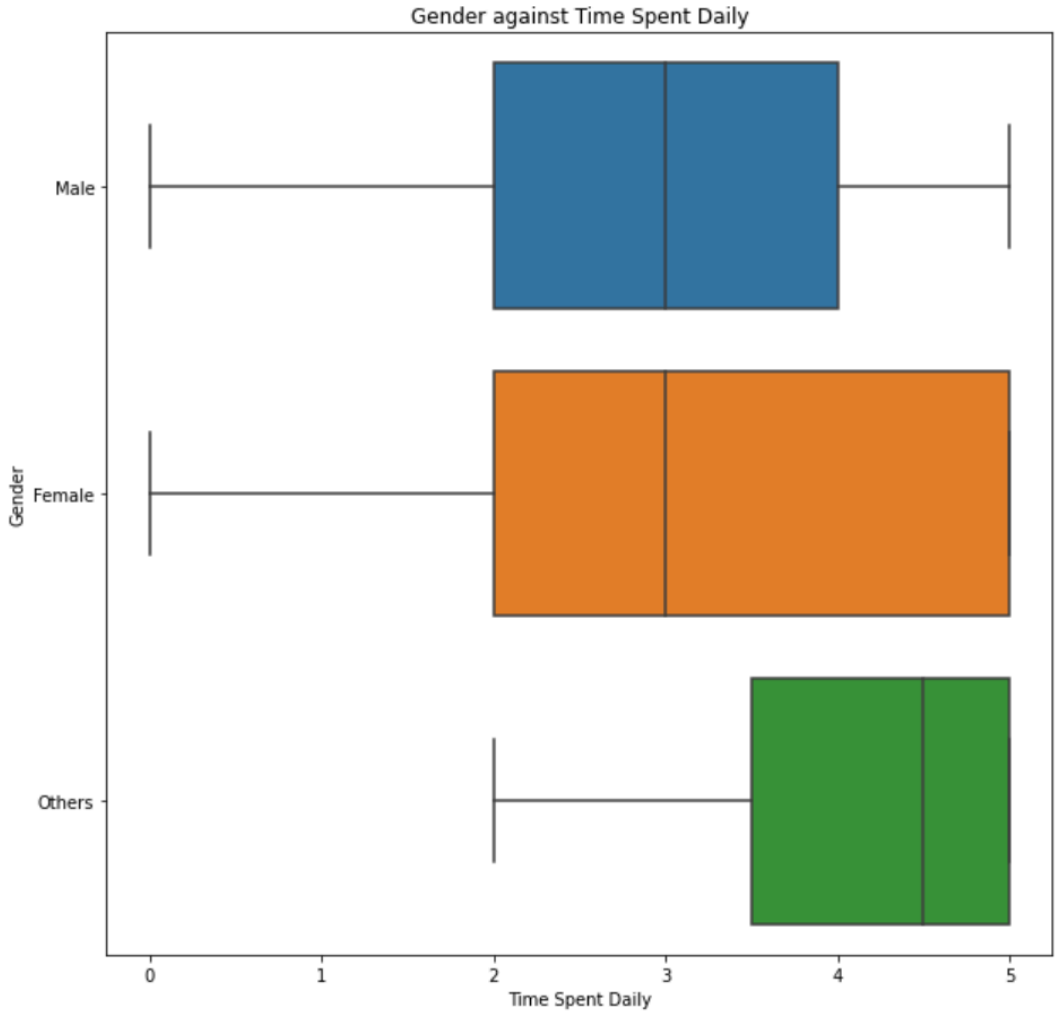
```
AxesSubplot(0.125,0.125;0.775x0.755)
```



A29

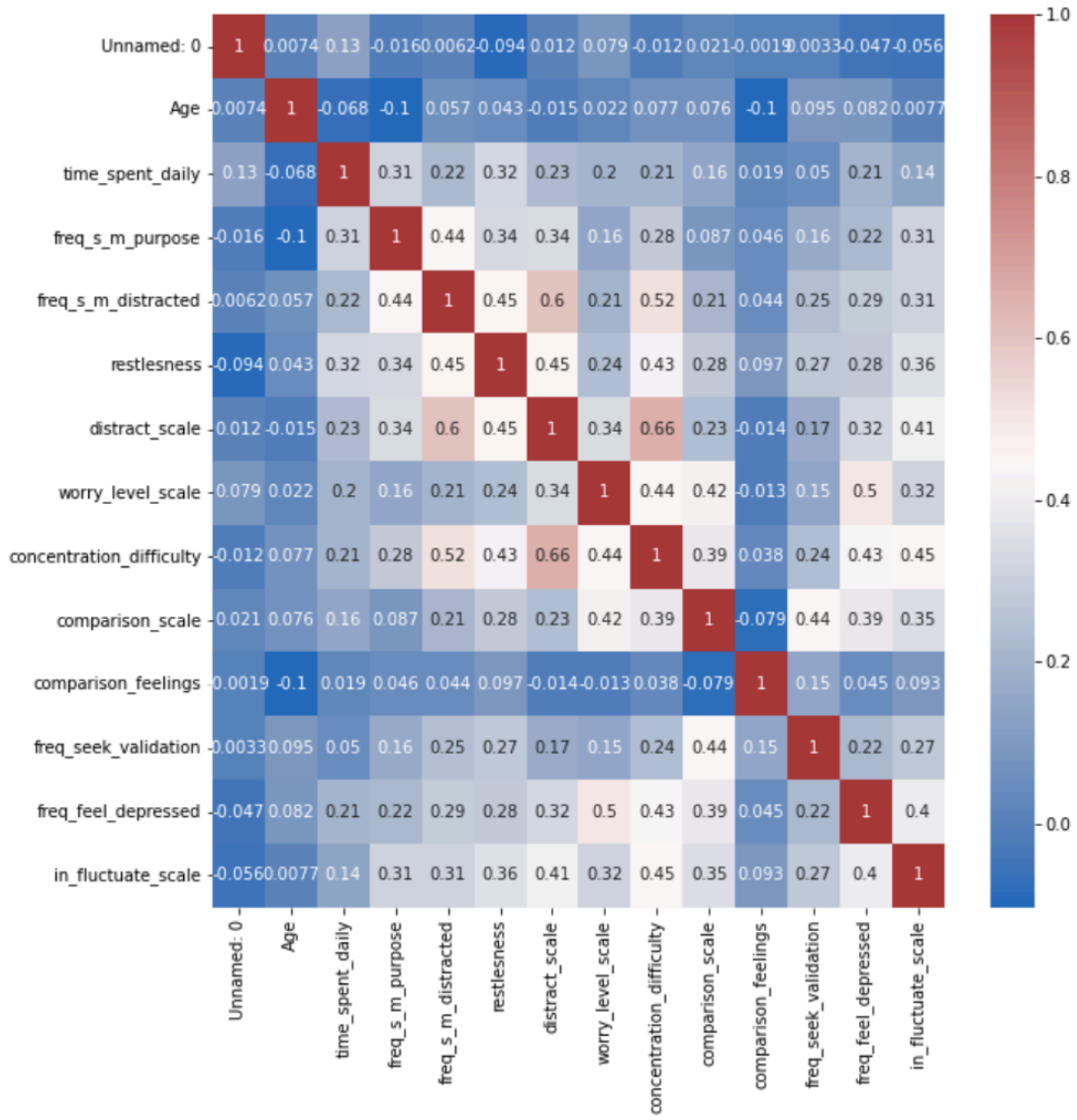
```
In [49]: #Does Gender affect the time spent on social media platforms?
         #Plot the boxplot
         plt.figure(figsize=(10,10))
         Gender_Time_Spent = sns.boxplot(data = new_social_media_dataset_df, x='time_spent_daily', y='Gender')
         # The title and the x and y-axis labels are added
         plt.xlabel('Time Spent Daily')
         plt.ylabel('Gender')
         plt.title('Gender against Time Spent Daily')
         #Display the boxplot
         print(Gender_Time_Spent)
```

```
AxesSubplot(0.125,0.125;0.775x0.755)
```



A30

In [50]:
```
#To view how closely related all the features are
plt.figure(figsize=(10,10))
#Load the heatmap to view the correlation between features
Book_correlation_matrix = sns.heatmap(new_social_media_dataset_df.corr(), cmap="vlag", annot = True)
#Display the heatmap
plt.show()
```



A31

In [51]:
```
#Normalise the time spent daily
Time_spent_daily_array = np.array(new_social_media_dataset_df['time_spent_daily'])
print("The Time spent daily array: ",Time_spent_daily_array)
Time_spent_daily_normalized = preprocessing.normalize([Time_spent_daily_array])
print("The Time spent daily normalized: ", Time_spent_daily_normalized)
```

```
The Time spent daily array:  [2 5 3 5 2 2 3 5 5 0 2 3 3 3 5 1 0 1 2 5 3 1 3 4 3 3 2 3 5 3 0 3 2 1 3 5 5
 3 0 5 5 2 2 1 5 1 5 1 3 2 1 3 2 5 2 3 3 3 5 5 4 2 1 2 2 2 3 5 4 5 4 5 4 2
 1 3 1 4 4 2 2 5 5 4 3 4 5 5 5 2 5 2 4 5 3 3 4 2 0 5 3 1 4 3 4 2 5 5 2 2 2
 4 5 0 1 3 5 3 2 3 3 5 0 4 1 5 2 1 3 5 5 3 2 5 5 0 1 3 5 2 4 3 0 3 1 5 2 5
 4 2 0 5 2 4 5 2 2 1 5 5 3 2 5 5 5 5 3 1 2 3 4 2 2 5 3 5 5 4 3 3 5 4 2 3 4
 2 1 4 2 1 2 2 2 3 4 3 5 3 3 5 5 1 5 5 3 5 5 5 5 5 0 5 5 2 2 2 3 5 4 5 3 3
 2 4 1 5 2 4 4 5 3 4 4 2 1 5 4 4 4 4 2 5 1 0 3 2 4 4 3 5 5 3 5 3 2 4 4 2 4
 5 5 5 5 5 5 5 2 3 5 3 5 4 5 3 5 3 5 4 3 3 3 3 1 5 0 5 1 3 3 5 5 5 4 4 2 5
 5 4 3 3 2 5 4 1 3 5 3 2 3 4 2 4 1 3 4 2 5 4 1 5 4 1 5 2]

The Time spent daily normalized:  [[0.03107224 0.07768059 0.04660836 0.07768059 0.03107224 0.03107224
  0.04660836 0.07768059 0.07768059 0.         0.03107224 0.04660836
  0.04660836 0.04660836 0.07768059 0.01553612 0.         0.01553612
  0.03107224 0.07768059 0.04660836 0.01553612 0.04660836 0.06214448
  0.04660836 0.04660836 0.03107224 0.04660836 0.07768059 0.04660836
  0.         0.04660836 0.03107224 0.01553612 0.04660836 0.07768059
  0.07768059 0.04660836 0.         0.07768059 0.07768059 0.03107224
  0.03107224 0.01553612 0.07768059 0.01553612 0.07768059 0.01553612
  0.04660836 0.03107224 0.01553612 0.04660836 0.03107224 0.07768059
  0.03107224 0.04660836 0.04660836 0.04660836 0.07768059 0.07768059
  0.06214448 0.03107224 0.01553612 0.03107224 0.03107224 0.03107224
  0.04660836 0.07768059 0.06214448 0.07768059 0.06214448 0.07768059
  0.06214448 0.03107224 0.01553612 0.04660836 0.01553612 0.06214448
  0.06214448 0.03107224 0.03107224 0.07768059 0.07768059 0.06214448
  0.04660836 0.06214448 0.07768059 0.07768059 0.07768059 0.03107224
  0.07768059 0.03107224 0.06214448 0.07768059 0.04660836 0.04660836
  0.06214448 0.03107224 0.         0.07768059 0.04660836 0.01553612
  0.06214448 0.04660836 0.06214448 0.03107224 0.07768059 0.07768059
  0.03107224 0.03107224 0.03107224 0.06214448 0.07768059 0.
  0.01553612 0.04660836 0.07768059 0.04660836 0.03107224 0.04660836
  0.04660836 0.07768059 0.         0.06214448 0.01553612 0.07768059
  0.03107224 0.01553612 0.04660836 0.07768059 0.07768059 0.04660836
  0.03107224 0.07768059 0.07768059 0.         0.01553612 0.04660836
  0.07768059 0.03107224 0.06214448 0.04660836 0.         0.04660836
  0.01553612 0.07768059 0.03107224 0.07768059 0.06214448 0.03107224
  0.         0.07768059 0.03107224 0.06214448 0.07768059 0.03107224
  0.03107224 0.01553612 0.07768059 0.07768059 0.04660836 0.03107224
  0.07768059 0.07768059 0.07768059 0.07768059 0.04660836 0.01553612
  0.03107224 0.04660836 0.06214448 0.03107224 0.03107224 0.07768059
  0.04660836 0.07768059 0.07768059 0.06214448 0.04660836 0.04660836
  0.07768059 0.06214448 0.03107224 0.04660836 0.06214448 0.03107224
  0.01553612 0.06214448 0.03107224 0.01553612 0.03107224 0.03107224
  0.03107224 0.04660836 0.06214448 0.04660836 0.07768059 0.04660836
  0.04660836 0.07768059 0.07768059 0.01553612 0.07768059 0.07768059
  0.04660836 0.07768059 0.07768059 0.07768059 0.07768059 0.07768059
  0.         0.07768059 0.07768059 0.03107224 0.03107224 0.03107224
  0.04660836 0.07768059 0.06214448 0.07768059 0.04660836 0.04660836
  0.03107224 0.06214448 0.01553612 0.07768059 0.03107224 0.06214448
  0.06214448 0.07768059 0.04660836 0.06214448 0.06214448 0.03107224
```

```
0.01553612 0.07768059 0.06214448 0.06214448 0.06214448 0.06214448
0.03107224 0.07768059 0.01553612 0.         0.04660836 0.03107224
0.06214448 0.06214448 0.04660836 0.07768059 0.07768059 0.04660836
0.07768059 0.04660836 0.03107224 0.06214448 0.06214448 0.03107224
0.06214448 0.07768059 0.07768059 0.07768059 0.07768059 0.07768059
0.07768059 0.07768059 0.03107224 0.04660836 0.07768059 0.04660836
0.07768059 0.06214448 0.07768059 0.04660836 0.07768059 0.04660836
0.07768059 0.06214448 0.04660836 0.04660836 0.04660836 0.04660836
0.01553612 0.07768059 0.         0.07768059 0.01553612 0.04660836
0.04660836 0.07768059 0.07768059 0.07768059 0.06214448 0.06214448
0.03107224 0.07768059 0.07768059 0.06214448 0.04660836 0.04660836
0.03107224 0.07768059 0.06214448 0.01553612 0.04660836 0.07768059
0.04660836 0.03107224 0.04660836 0.06214448 0.03107224 0.06214448
0.01553612 0.04660836 0.06214448 0.03107224 0.07768059 0.06214448
0.01553612 0.07768059 0.06214448 0.01553612 0.07768059 0.03107224]]
```

A32

In [53]:

```python
from sklearn.preprocessing import OneHotEncoder

# Converting the type of columns gender and risk categery to category
new_social_media_dataset_df['Gender'] = new_social_media_dataset_df['Gender'].astype('category')
new_social_media_dataset_df['Risk Category'] = new_social_media_dataset_df['Risk Category'].astype('category')

#Assign numbers to the two features and store them in a new column
new_social_media_dataset_df['Gender_Num'] = new_social_media_dataset_df['Gender'].cat.codes
new_social_media_dataset_df['Risk_catogeries_Num'] = new_social_media_dataset_df['Risk Category'].cat.codes

# Assign a name value to the encoder
Encoder = OneHotEncoder()

# Fit the columns into the encoder
Encoder_new_data = pd.DataFrame(Encoder.fit_transform(new_social_media_dataset_df[['Gender_Num', 'Risk_catogeries_Num']]).toa

# Merge the new columns with our dataset
New_social_media_data_frame = new_social_media_dataset_df.join(Encoder_new_data)

#Display the datset with the new columns
print(New_social_media_data_frame)
```

```
     in_fluctuate_scale  Risk Category  Gender_Num  Risk_catogeries_Num    0  \
0                     4           Safe           1                    2  0.0
1                     4      High Risk           0                    0  1.0
2                     2       Moderate           0                    1  1.0
3                     3      High Risk           0                    0  1.0
4                     4           Safe           0                    2  1.0
..                  ...            ...         ...                  ...  ...
319                   2      High Risk           1                    0  0.0
320                   1       Moderate           1                    1  0.0
321                   4           Safe           0                    2  1.0
322                   3      High Risk           0                    0  1.0
323                   5           Safe           1                    2  0.0

       1    2    3    4    5
0    1.0  0.0  0.0  0.0  1.0
1    0.0  0.0  1.0  0.0  0.0
2    0.0  0.0  0.0  1.0  0.0
3    0.0  0.0  1.0  0.0  0.0
4    0.0  0.0  0.0  0.0  1.0
..   ...  ...  ...  ...  ...
319  1.0  0.0  1.0  0.0  0.0
320  1.0  0.0  0.0  1.0  0.0
321  0.0  0.0  0.0  0.0  1.0
322  0.0  0.0  1.0  0.0  0.0
323  1.0  0.0  0.0  0.0  1.0

[324 rows x 30 columns]
```

A33

In [54]:

```
#Normalise the Gender and Risk catogery
Gender_array = np.array(new_social_media_dataset_df['Gender_Num'])
print("Gender array: ",Gender_array)
Gender_normalized = preprocessing.normalize([Gender_array])
print("Gender normalized: ", Gender_normalized)


Risk_catogery_array = np.array(new_social_media_dataset_df['Risk_catogeries_Num'])
print("Risk_catogery array: ",Risk_catogery_array)
Risk_catogery_normalized = preprocessing.normalize([Risk_catogery_array])
print("Risk_catogery normalized: ", Risk_catogery_normalized)
```

```
Gender array:  [1 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 1 1 1 0 0 1 0 0 0 0 0 2 1 1 0 1 0 0 0 0
 0 1 1 1 0 0 0 0 0 0 1 1 2 1 1 1 0 1 0 0 1 1 1 0 1 0 1 0 1 0 0 2 1 1 0 1 0
 1 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 1 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 0
 0 0 1 0 1 0 1 1 1 0 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 1 0 0 0 0 0
 1 0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 1 0 1 0 0 0
 0 0 1 0 1 0 1 1 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 1 2 1 0 1 1 0 0 0 1 0
 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 0 1 0 0 1 0 0 0 0 0 0 1 1 0 0 1 0 1 1 1 1 0 0
 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 1 0 1 0 0 1 1 1 1 1 1 1 0 0 1 1 0 1 0 0 0
 1 1 1 0 1 0 1 1 0 0 1 0 0 0 0 0 0 1 0 1 1 1 1 1 1 1 0 0 1]
Gender normalized:  [[0.08219949 0.         0.         0.         0.         0.
  0.         0.         0.         0.08219949 0.08219949 0.
  0.         0.         0.08219949 0.08219949 0.08219949 0.08219949
  0.08219949 0.08219949 0.         0.         0.08219949 0.
  0.         0.         0.         0.         0.16439899 0.08219949
  0.08219949 0.         0.08219949 0.         0.         0.
  0.         0.         0.08219949 0.08219949 0.08219949 0.
  0.         0.         0.         0.         0.         0.08219949
  0.08219949 0.16439899 0.08219949 0.08219949 0.08219949 0.
  0.08219949 0.         0.         0.08219949 0.08219949 0.08219949
  0.         0.08219949 0.         0.08219949 0.         0.08219949
  0.         0.         0.16439899 0.08219949 0.08219949 0.
  0.08219949 0.         0.08219949 0.08219949 0.08219949 0.
  0.         0.         0.         0.         0.         0.08219949
  0.         0.         0.         0.         0.         0.08219949
  0.         0.08219949 0.         0.         0.08219949 0.
  0.         0.08219949 0.08219949 0.08219949 0.         0.08219949
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.08219949
  0.         0.08219949 0.         0.08219949 0.08219949 0.08219949
  0.         0.08219949 0.08219949 0.08219949 0.08219949 0.
```

```
0.08219949 0.         0.08219949 0.08219949 0.08219949 0.
0.         0.         0.         0.         0.         0.08219949
0.         0.         0.         0.         0.         0.08219949
0.         0.08219949 0.         0.         0.08219949 0.
0.         0.08219949 0.08219949 0.08219949 0.         0.08219949
0.         0.         0.         0.         0.         0.
0.         0.         0.         0.         0.         0.08219949
0.         0.08219949 0.         0.08219949 0.08219949 0.08219949
0.         0.08219949 0.08219949 0.08219949 0.08219949 0.
0.         0.         0.         0.         0.         0.
0.         0.08219949 0.         0.         0.08219949 0.
0.08219949 0.         0.         0.08219949 0.08219949 0.
0.         0.         0.         0.         0.08219949 0.
0.08219949 0.         0.         0.         0.         0.
0.         0.         0.08219949 0.         0.         0.08219949
0.         0.         0.         0.         0.         0.08219949
0.08219949 0.         0.08219949 0.         0.08219949 0.08219949
0.08219949 0.         0.08219949 0.         0.         0.08219949
0.         0.08219949 0.         0.         0.         0.
0.         0.08219949 0.         0.08219949 0.         0.08219949
0.08219949 0.         0.         0.         0.         0.
0.         0.08219949 0.         0.08219949 0.         0.
0.08219949 0.         0.         0.         0.         0.
0.08219949 0.08219949 0.16439899 0.08219949 0.         0.08219949
0.08219949 0.         0.         0.         0.08219949 0.
0.08219949 0.08219949 0.08219949 0.         0.         0.
0.08219949 0.         0.         0.08219949 0.08219949 0.
0.08219949 0.         0.08219949 0.         0.08219949 0.
0.         0.08219949 0.         0.         0.         0.
0.         0.08219949 0.08219949 0.         0.         0.08219949
0.         0.08219949 0.08219949 0.08219949 0.08219949 0.
0.         0.08219949 0.08219949 0.08219949 0.         0.
0.         0.         0.         0.08219949 0.08219949 0.
0.         0.         0.         0.         0.08219949 0.
0.08219949 0.         0.08219949 0.         0.         0.08219949
0.08219949 0.08219949 0.08219949 0.08219949 0.08219949 0.
0.         0.08219949 0.08219949 0.         0.08219949 0.
0.         0.         0.08219949 0.08219949 0.08219949 0.
0.08219949 0.         0.08219949 0.08219949 0.         0.
0.08219949 0.         0.         0.         0.         0.
0.08219949 0.         0.08219949 0.08219949 0.08219949 0.08219949
0.08219949 0.08219949 0.08219949 0.         0.         0.08219949]]
```

```
Risk_catogery array:  [2 0 1 0 2 2 1 0 0 2 2 1 1 1 0 2 2 2 2 0 1 2 1 1 1 1 2 1 0 1 2 1 2 2 1 0 0
 1 2 0 0 2 2 2 0 2 0 2 1 2 2 1 2 0 2 1 1 1 0 0 1 2 2 2 2 2 1 0 1 0 1 0 1 2
 2 1 2 1 1 2 2 0 0 1 1 1 0 0 0 2 0 2 1 0 1 1 1 2 2 0 1 2 1 1 1 2 0 0 2 2 2
 1 0 2 2 1 0 1 2 1 1 0 2 1 2 0 2 2 1 0 0 1 2 0 0 2 2 1 0 2 1 1 2 1 2 0 2 0
 1 2 2 0 2 1 0 2 2 2 0 0 1 2 0 0 0 0 1 2 2 1 1 2 2 0 1 0 0 1 1 1 0 1 2 1 1
 2 2 1 2 2 2 2 2 1 1 1 0 1 1 0 0 2 0 0 1 0 0 0 0 0 0 2 0 0 2 2 2 1 0 1 0 1 1
 2 1 2 0 2 1 1 0 1 1 1 2 2 0 1 1 1 1 2 0 2 2 1 2 1 1 1 0 0 1 0 1 2 1 1 2 1
 0 0 0 0 0 0 0 2 1 0 1 0 1 0 1 0 1 0 1 1 1 1 1 2 0 2 0 2 1 1 0 0 0 1 1 2 0
 0 1 1 1 2 0 1 2 1 0 1 2 1 1 2 1 2 1 1 2 0 1 2 0 1 2 0 2]
Risk_catogery normalized:  [[0.08567059 0.         0.04283529 0.         0.08567059 0.08567059
  0.04283529 0.         0.         0.08567059 0.08567059 0.04283529
  0.04283529 0.04283529 0.         0.08567059 0.08567059 0.08567059
  0.08567059 0.         0.04283529 0.08567059 0.04283529 0.04283529
  0.04283529 0.04283529 0.08567059 0.04283529 0.         0.04283529
  0.08567059 0.04283529 0.08567059 0.08567059 0.04283529 0.
  0.         0.04283529 0.08567059 0.         0.         0.08567059
  0.08567059 0.08567059 0.         0.08567059 0.         0.08567059
  0.04283529 0.08567059 0.08567059 0.04283529 0.08567059 0.
  0.08567059 0.04283529 0.04283529 0.04283529 0.         0.
  0.04283529 0.08567059 0.08567059 0.08567059 0.08567059 0.08567059
  0.04283529 0.         0.04283529 0.         0.04283529 0.
  0.04283529 0.08567059 0.08567059 0.04283529 0.08567059 0.04283529
  0.04283529 0.08567059 0.08567059 0.         0.         0.04283529
  0.04283529 0.04283529 0.         0.         0.         0.08567059
  0.         0.08567059 0.04283529 0.         0.04283529 0.04283529
  0.04283529 0.08567059 0.08567059 0.         0.04283529 0.08567059
  0.04283529 0.04283529 0.04283529 0.08567059 0.         0.
  0.08567059 0.08567059 0.08567059 0.04283529 0.         0.08567059
  0.08567059 0.04283529 0.         0.04283529 0.08567059 0.04283529
  0.04283529 0.         0.08567059 0.04283529 0.08567059 0.
  0.08567059 0.08567059 0.04283529 0.         0.         0.04283529
```

```
0.08567059 0.08567059 0.08567059 0.04283529 0.         0.08567059
0.08567059 0.04283529 0.         0.04283529 0.08567059 0.04283529
0.04283529 0.         0.08567059 0.04283529 0.08567059 0.
0.08567059 0.08567059 0.04283529 0.         0.         0.04283529
0.08567059 0.         0.         0.08567059 0.08567059 0.04283529
0.         0.08567059 0.04283529 0.04283529 0.08567059 0.04283529
0.08567059 0.         0.08567059 0.         0.04283529 0.08567059
0.08567059 0.         0.08567059 0.04283529 0.         0.08567059
0.08567059 0.08567059 0.         0.         0.04283529 0.08567059
0.         0.         0.         0.         0.04283529 0.08567059
0.08567059 0.04283529 0.04283529 0.08567059 0.08567059 0.
0.04283529 0.         0.         0.04283529 0.04283529 0.04283529
0.         0.04283529 0.08567059 0.04283529 0.04283529 0.08567059
0.08567059 0.04283529 0.08567059 0.08567059 0.08567059 0.08567059
0.08567059 0.04283529 0.04283529 0.04283529 0.         0.04283529
0.04283529 0.         0.         0.08567059 0.         0.
0.04283529 0.         0.         0.         0.         0.
0.08567059 0.         0.         0.08567059 0.08567059 0.08567059
0.04283529 0.         0.04283529 0.         0.04283529 0.04283529
0.08567059 0.04283529 0.08567059 0.         0.08567059 0.04283529
0.04283529 0.         0.04283529 0.04283529 0.04283529 0.08567059
0.08567059 0.         0.04283529 0.04283529 0.04283529 0.04283529
0.08567059 0.         0.08567059 0.08567059 0.04283529 0.08567059
0.04283529 0.04283529 0.04283529 0.         0.         0.04283529
0.         0.04283529 0.08567059 0.04283529 0.04283529 0.08567059
0.04283529 0.         0.         0.         0.         0.
0.         0.         0.08567059 0.04283529 0.         0.04283529
0.         0.04283529 0.         0.04283529 0.         0.04283529
0.         0.04283529 0.04283529 0.04283529 0.04283529 0.04283529
0.08567059 0.         0.08567059 0.         0.08567059 0.04283529
0.04283529 0.         0.         0.         0.04283529 0.04283529
0.08567059 0.         0.         0.04283529 0.04283529 0.04283529
0.08567059 0.         0.04283529 0.08567059 0.04283529 0.
0.04283529 0.08567059 0.04283529 0.04283529 0.08567059 0.04283529
0.08567059 0.04283529 0.04283529 0.08567059 0.         0.04283529
0.08567059 0.         0.04283529 0.08567059 0.         0.08567059]]
```

A34

In [55]:
```
import tensorflow as tf
# convert it to tensorflow
tensor_TS = tf.convert_to_tensor(Time_spent_daily_array,name='tensor_TS')
print(tensor_TS)
```

```
tf.Tensor(
[2 5 3 5 2 2 3 5 5 0 2 3 3 3 5 1 0 1 2 5 3 1 3 4 3 3 2 3 5 3 0 3 2 1 3 5 5
 3 0 5 5 2 2 1 5 1 5 1 3 2 1 3 2 5 2 3 3 3 5 5 4 2 1 2 2 2 3 5 4 5 4 5 4 2
 1 3 1 4 4 2 2 5 5 4 3 4 5 5 5 2 5 2 4 5 3 3 4 2 0 5 3 1 4 3 4 2 5 5 2 2 2
 4 5 0 1 3 5 3 2 3 3 5 0 4 1 5 2 1 3 5 5 3 2 5 5 0 1 3 5 2 4 3 0 3 1 5 2 5
 4 2 0 5 2 4 5 2 2 1 5 5 3 2 5 5 5 5 3 1 2 3 4 2 2 5 3 5 5 4 3 3 5 4 2 3 4
 2 1 4 2 1 2 2 2 3 4 3 5 3 3 5 5 1 5 5 3 5 5 5 5 5 5 0 5 5 2 2 2 3 5 4 5 3 3
 2 4 1 5 2 4 4 5 3 4 4 2 1 5 4 4 4 4 2 5 1 0 3 2 4 4 3 5 5 3 5 3 2 4 4 2 4
 5 5 5 5 5 5 5 2 3 5 3 5 4 5 3 5 3 5 4 3 3 3 3 1 5 0 5 1 3 3 5 5 5 4 4 2 5
 5 4 3 3 2 5 4 1 3 5 3 2 3 4 2 4 1 3 4 2 5 4 1 5 4 1 5 2], shape=(324,), dtype=int64)
```

A35

In [56]:
```
#convert Gender and Risk catogery to tensors
tensor_Gender = tf.convert_to_tensor(Gender_array,name='tensor_Gender')
print(tensor_Gender)

tensor_RC = tf.convert_to_tensor(Risk_catogery_array,name='tensor_RC')
print(tensor_RC)
```

```
tf.Tensor(
[1 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 1 1 1 0 0 1 0 0 0 0 0 2 1 1 0 1 0 0 0 0
 0 1 1 1 0 0 0 0 0 0 1 1 2 1 1 1 0 1 0 0 1 1 1 0 1 0 1 0 1 0 0 2 1 1 0 1 0
 1 1 1 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 1 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 0
 0 0 1 0 1 0 1 1 1 0 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 1 1 0 0 0 0 0
 1 0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 1 0 1 0 0 0
 0 0 1 0 1 0 1 1 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 1 2 1 0 1 1 0 0 0 1 0
 1 1 1 0 0 0 1 0 0 1 1 0 1 0 1 0 1 0 0 1 0 0 0 0 0 0 1 1 0 0 1 0 1 1 1 1 1 0 0
 1 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 1 0 1 0 0 1 1 1 1 1 1 1 0 0 1 1 0 1 0 0 0
 1 1 1 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 1 0 1 1 1 1 1 1 1 1 0 0 1], shape=(324,), dtype=int8)
tf.Tensor(
[2 0 1 0 2 2 1 0 0 2 2 1 1 1 0 2 2 2 2 0 1 2 1 1 1 1 2 1 0 1 2 1 2 2 1 0 0
 1 2 0 0 2 2 2 0 2 0 2 1 2 2 1 2 0 2 1 1 1 0 0 1 2 2 2 2 2 1 0 1 0 1 0 1 2
 2 1 2 1 1 2 2 0 0 1 1 1 0 0 0 2 0 2 1 0 1 1 1 2 2 0 1 2 1 1 1 2 0 0 2 2 2
 1 0 2 2 1 0 1 2 1 1 0 2 1 2 0 2 2 1 0 0 1 2 0 0 2 2 1 0 2 1 1 2 1 2 0 2 0
 1 2 2 0 2 1 0 2 2 2 0 0 1 2 0 0 0 0 1 2 2 1 1 2 2 0 1 0 0 1 1 1 0 1 2 1 1
 2 2 1 2 2 2 2 2 1 1 1 0 1 1 0 0 2 0 0 1 0 0 0 0 0 0 2 0 0 2 2 2 1 0 1 0 1 1
 2 1 2 0 2 1 1 0 1 1 1 2 2 0 1 1 1 1 2 0 2 2 1 2 1 1 1 0 0 1 0 1 2 1 1 2 1
 0 0 0 0 0 0 0 2 1 0 1 0 1 0 1 0 1 0 1 1 1 1 1 2 0 2 0 2 1 1 0 0 0 1 1 2 0
 0 1 1 1 2 0 1 2 1 0 1 2 1 1 2 1 2 1 1 2 0 1 2 0 1 2 0 2], shape=(324,), dtype=int8)
```

A36

In [58]:
```
#Training, Validation and test sets are generateed
TS_X = Time_spent_daily_normalized.reshape(-1,1)
TS_Y = Risk_catogery_normalized.reshape(-1,1)
TS_x_train,TS_y_train,TS_x_valid,TS_y_valid,TS_x_test, TS_y_test = train_valid_test_split(TS_X,TS_Y,test_size=0.25)
```

```
TS_x_val = TS_x_train[:100]
TS_half_x_train = TS_x_train[100:]

TS_y_val = TS_y_train[:100]
TS_half_y_train = TS_y_train[100:]
```

A37

In [62]:
```
#Building the baseline model
RF_model = RandomForestClassifier(n_estimators=60, random_state=24, criterion='entropy', min_samples_split=3)
#Train the Random Forest model
RF_model.fit(TS_x_train, TS_y_train)
#F1 Score for Random Forest Model
Predicted_RF_Y = RF_model.predict(TS_x_test)
F1_score_RF = RF_model.score(TS_y_test,Predicted_RF_Y)
print(F1_score_RF)
```

```
0.27
```

A38

Simple Machine Learning Model

In [65]:
```
#Build a simple machine learning model
TS_simple_model = models.Sequential()
Ts_simple_model.add(layers.Flatten(input_shape=(100))
TS_simple_model.add(layers.Dense(64, activation='relu', input_shape=(100,)))
TS_simple_model.add(layers.Dense(64, activation='relu'))
TS_simple_model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
TS_simple_hist = model.fit(TS_half_x_train,
                    TS_half_y_train,
                    epochs=20,
                    batch_size=512,
                    validation_data=(TS_x_val, TS_y_val))
```

```
Epoch 1/20
30/30 [==============================] - 28s 494ms/step - loss: 2.1684 - accuracy: 0.0015 - val_loss: 1.9720 - val_accuracy:
0.0060
Epoch 2/20
30/30 [==============================] - 2s 50ms/step - loss: 1.8582 - accuracy: 0.0119 - val_loss: 1.8381 - val_accuracy:
0.0166
Epoch 3/20
30/30 [==============================] - 1s 45ms/step - loss: 1.7338 - accuracy: 0.0176 - val_loss: 1.7682 - val_accuracy:
0.0243
Epoch 4/20
30/30 [==============================] - 2s 53ms/step - loss: 1.6585 - accuracy: 0.0211 - val_loss: 1.7483 - val_accuracy:
0.0035
Epoch 5/20
30/30 [==============================] - 2s 53ms/step - loss: 1.5711 - accuracy: 0.0150 - val_loss: 1.3480 - val_accuracy:
0.0065
Epoch 6/20
30/30 [==============================] - 2s 56ms/step - loss: 1.1599 - accuracy: 0.0043 - val_loss: 1.3272 - val_accuracy:
0.0043
Epoch 7/20
30/30 [==============================] - 2s 52ms/step - loss: 1.1122 - accuracy: 0.0057 - val_loss: 1.3535 - val_accuracy:
0.0091
Epoch 8/20
30/30 [==============================] - 1s 44ms/step - loss: 1.0845 - accuracy: 0.0096 - val_loss: 1.3854 - val_accuracy:
0.0203
Epoch 9/20
30/30 [==============================] - 1s 46ms/step - loss: 1.0634 - accuracy: 0.0150 - val_loss: 1.4005 - val_accuracy:
0.0254
Epoch 10/20
30/30 [==============================] - 2s 51ms/step - loss: 1.0452 - accuracy: 0.0301 - val_loss: 1.4327 - val_accuracy:
0.0635
```

```
Epoch 11/20
30/30 [==============================] - 2s 54ms/step - loss: 1.0305 - accuracy: 0.1118 - val_loss: 1.5372 - val_accuracy:
0.1275
Epoch 12/20
30/30 [==============================] - 1s 42ms/step - loss: 0.9141 - accuracy: 0.0699 - val_loss: 1.0366 - val_accuracy:
0.0049
Epoch 13/20
30/30 [==============================] - 1s 44ms/step - loss: 0.5801 - accuracy: 0.0227 - val_loss: 1.1136 - val_accuracy:
0.0465
Epoch 14/20
30/30 [==============================] - 2s 51ms/step - loss: 0.5563 - accuracy: 0.0521 - val_loss: 1.1872 - val_accuracy:
0.0501
Epoch 15/20
30/30 [==============================] - 1s 47ms/step - loss: 0.5395 - accuracy: 0.0416 - val_loss: 1.1970 - val_accuracy:
0.0307
Epoch 16/20
30/30 [==============================] - 1s 50ms/step - loss: 0.5260 - accuracy: 0.0828 - val_loss: 1.2138 - val_accuracy:
0.1532
Epoch 17/20
30/30 [==============================] - 2s 53ms/step - loss: 0.5174 - accuracy: 0.2251 - val_loss: 1.2828 - val_accuracy:
0.2150
Epoch 18/20
30/30 [==============================] - 1s 44ms/step - loss: 0.5123 - accuracy: 0.2930 - val_loss: 1.3195 - val_accuracy:
0.2601
Epoch 19/20
30/30 [==============================] - 2s 51ms/step - loss: 0.5055 - accuracy: 0.3223 - val_loss: 1.3364 - val_accuracy:
0.2997
Epoch 20/20
30/30 [==============================] - 1s 49ms/step - loss: 0.5008 - accuracy: 0.3679 - val_loss: 1.3841 - val_accuracy:
0.3202
```

A39

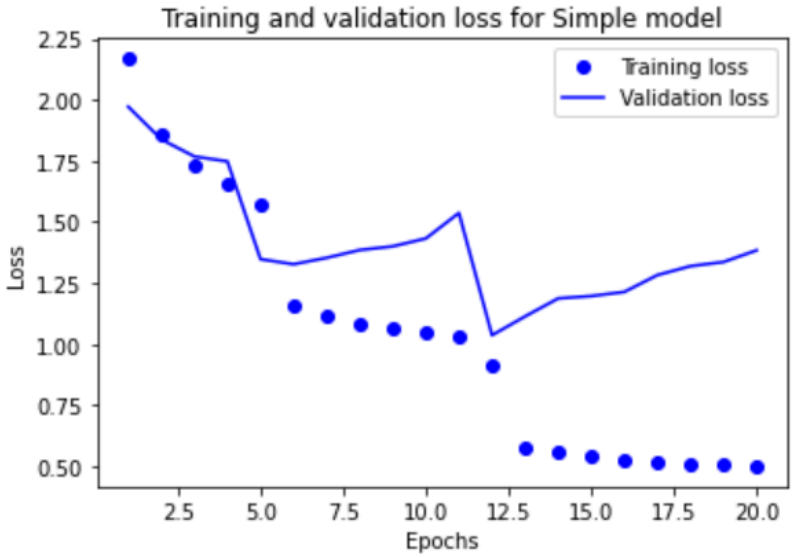
In [66]:

```
#Plot the Training and Validation loss for Simple machine learning Model
TS_simple_hist_dict = TS_simple_hist.history
TS_simple_model_loss_values = TS_simple_hist_dict['loss']
TS_simple_model_val_loss_values = TS_simple_hist_dict['val_loss']

TS_simple_epochs = range(1, len(TS_simple_hist_dict['loss']) + 1)

plt.plot(TS_simple_epochs, TS_simple_model_loss_values, 'bo', label='Training loss')
plt.plot(TS_simple_epochs, TS_simple_model_val_loss_values, 'b', label='Validation loss')
plt.title('Training and validation loss for Simple Model')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



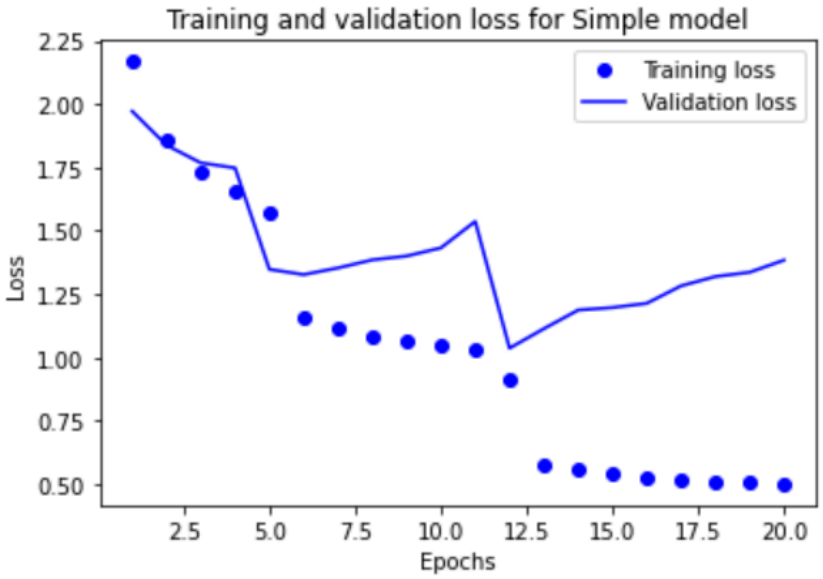
A40

In [67]:

```
#Plot the Training and Validation accuracy for Simple machine Learning Model
plt.clf()
TS_simple_model_accuracy_values = TS_simple_hist_dict['acc']
TS_simple_model_val_acc_values = TS_simple_hist_dict['val_acc']

plt.plot(TS_simple_epochs, TS_simple_model_accuracy_values, 'bo', label='Training acc')
plt.plot(TS_simple_epochs, TS_simple_model_val_acc_values, 'b', label='Validation acc')
plt.title('Training and validation accuracy for Simple Model')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



A41

Recurent Neural Network(RNN)

In [69]:

```
#Built a RNN model
TS_RNN_model = Sequential()
TS_RNN_model.add(layers.LSTM(32, input_shape=(100,))
TS_RNN_model.add(layers.Dense(1))

TS_RNN_model.compile(optimizer=RMSprop(), loss='accuracy')
TS_RNN_model_hist = TS_RNN_model.fit(TS_x_train,TS_y_train,
                                     steps_per_epoch=100,
                                     epochs=50,
                                     validation_data=(TS_x_val, TS_y_val))
```

```
Epoch 1/50
30/30 [==============================] - 17s 504ms/step - loss: 1.3547 - acc: 0.2481 - val_loss: 0.7881 - val_acc: 0.2093
Epoch 2/50
30/30 [==============================] - 2s 62ms/step - loss: 0.6652 - acc: 0.2909 - val_loss: 0.5885 - val_acc: 0.3170
Epoch 3/50
30/30 [==============================] - 2s 73ms/step - loss: 0.4768 - acc: 0.2842 - val_loss: 0.4383 - val_acc: 0.1998
Epoch 4/50
30/30 [==============================] - 2s 68ms/step - loss: 0.3516 - acc: 0.1956 - val_loss: 0.3735 - val_acc: 0.1424
Epoch 5/50
30/30 [==============================] - 2s 79ms/step - loss: 0.2624 - acc: 0.2367 - val_loss: 0.3933 - val_acc: 0.3258
Epoch 6/50
30/30 [==============================] - 2s 73ms/step - loss: 0.2027 - acc: 0.3507 - val_loss: 0.4296 - val_acc: 0.2406
Epoch 7/50
30/30 [==============================] - 2s 64ms/step - loss: 0.1670 - acc: 0.3315 - val_loss: 0.4590 - val_acc: 0.3733
Epoch 8/50
30/30 [==============================] - 2s 64ms/step - loss: 0.1383 - acc: 0.3935 - val_loss: 0.5350 - val_acc: 0.3505
Epoch 9/50
30/30 [==============================] - 2s 70ms/step - loss: 0.1100 - acc: 0.4060 - val_loss: 0.5925 - val_acc: 0.3870
Epoch 10/50
```

```
Epoch 10/50
30/30 [==============================] - 2s 65ms/step - loss: 0.0916 - acc: 0.4459 - val_loss: 0.8097 - val_acc: 0.4235
Epoch 11/50
30/30 [==============================] - 2s 59ms/step - loss: 0.0778 - acc: 0.4749 - val_loss: 0.7411 - val_acc: 0.4622
Epoch 12/50
30/30 [==============================] - 2s 54ms/step - loss: 0.0670 - acc: 0.5083 - val_loss: 0.8165 - val_acc: 0.4969
Epoch 13/50
30/30 [==============================] - 2s 57ms/step - loss: 0.0549 - acc: 0.5457 - val_loss: 0.9000 - val_acc: 0.4357
Epoch 14/50
30/30 [==============================] - 1s 50ms/step - loss: 0.0476 - acc: 0.5625 - val_loss: 0.9638 - val_acc: 0.4719
Epoch 15/50
30/30 [==============================] - 1s 50ms/step - loss: 0.0438 - acc: 0.5892 - val_loss: 1.0070 - val_acc: 0.4914
Epoch 16/50
30/30 [==============================] - 1s 49ms/step - loss: 0.0404 - acc: 0.6003 - val_loss: 1.0341 - val_acc: 0.4959
Epoch 17/50
30/30 [==============================] - 2s 55ms/step - loss: 0.0394 - acc: 0.6034 - val_loss: 1.0404 - val_acc: 0.6489
Epoch 18/50
30/30 [==============================] - 2s 58ms/step - loss: 0.0243 - acc: 0.6745 - val_loss: 1.1145 - val_acc: 0.4281
Epoch 19/50
30/30 [==============================] - 2s 55ms/step - loss: 0.0351 - acc: 0.6364 - val_loss: 1.0887 - val_acc: 0.6528
```

```
Epoch 19/50
30/30 [==============================] - 2s 55ms/step - loss: 0.0351 - acc: 0.6364 - val_loss: 1.0887 - val_acc: 0.6528
Epoch 20/50
30/30 [==============================] - 2s 53ms/step - loss: 0.0217 - acc: 0.7689 - val_loss: 1.1507 - val_acc: 0.5469
Epoch 21/50
30/30 [==============================] - 2s 52ms/step - loss: 0.0202 - acc: 0.6804 - val_loss: 1.2789 - val_acc: 0.5196
Epoch 22/50
30/30 [==============================] - 2s 52ms/step - loss: 0.0343 - acc: 0.6438 - val_loss: 1.1875 - val_acc: 0.5368
Epoch 23/50
30/30 [==============================] - 1s 50ms/step - loss: 0.0191 - acc: 0.6333 - val_loss: 1.2385 - val_acc: 0.4934
Epoch 24/50
30/30 [==============================] - 1s 50ms/step - loss: 0.0365 - acc: 0.6025 - val_loss: 1.1929 - val_acc: 0.4843
Epoch 25/50
30/30 [==============================] - 2s 56ms/step - loss: 0.0189 - acc: 0.6081 - val_loss: 1.2376 - val_acc: 0.4723
Epoch 26/50
30/30 [==============================] - 2s 52ms/step - loss: 0.0184 - acc: 0.6387 - val_loss: 1.2338 - val_acc: 0.4978
Epoch 27/50
30/30 [==============================] - 2s 64ms/step - loss: 0.0398 - acc: 0.6177 - val_loss: 1.2494 - val_acc: 0.4765
Epoch 28/50
30/30 [==============================] - 1s 49ms/step - loss: 0.0181 - acc: 0.6159 - val_loss: 1.2643 - val_acc: 0.5156
```

```
Epoch 28/50
30/30 [==============================] - 1s 49ms/step - loss: 0.0181 - acc: 0.6159 - val_loss: 1.2643 - val_acc: 0.5156
Epoch 29/50
30/30 [==============================] - 1s 45ms/step - loss: 0.0336 - acc: 0.5731 - val_loss: 1.2487 - val_acc: 0.4355
Epoch 30/50
30/30 [==============================] - 1s 48ms/step - loss: 0.0176 - acc: 0.5571 - val_loss: 1.2720 - val_acc: 0.4374
Epoch 31/50
30/30 [==============================] - 2s 67ms/step - loss: 0.0333 - acc: 0.5419 - val_loss: 1.2797 - val_acc: 0.3626
Epoch 32/50
30/30 [==============================] - 2s 51ms/step - loss: 0.0173 - acc: 0.4913 - val_loss: 1.2873 - val_acc: 0.3980
Epoch 33/50
30/30 [==============================] - 2s 57ms/step - loss: 0.0166 - acc: 0.5207 - val_loss: 1.2886 - val_acc: 0.4141
Epoch 34/50
30/30 [==============================] - 2s 60ms/step - loss: 0.0165 - acc: 0.5571 - val_loss: 1.2987 - val_acc: 0.4409
Epoch 35/50
30/30 [==============================] - 2s 58ms/step - loss: 0.0244 - acc: 0.5619 - val_loss: 1.2770 - val_acc: 0.4721
Epoch 36/50
30/30 [==============================] - 2s 68ms/step - loss: 0.0165 - acc: 0.5918 - val_loss: 1.2867 - val_acc: 0.4797
Epoch 37/50
```

```
Epoch 37/50
30/30 [==============================] - 2s 57ms/step - loss: 0.0324 - acc: 0.5403 - val_loss: 1.2761 - val_acc: 0.4052
Epoch 38/50
30/30 [==============================] - 2s 55ms/step - loss: 0.0167 - acc: 0.5321 - val_loss: 1.3151 - val_acc: 0.4158
Epoch 39/50
30/30 [==============================] - 1s 49ms/step - loss: 0.0163 - acc: 0.5414 - val_loss: 1.3141 - val_acc: 0.4504
Epoch 40/50
30/30 [==============================] - 2s 54ms/step - loss: 0.0276 - acc: 0.5599 - val_loss: 1.3221 - val_acc: 0.4321
Epoch 41/50
30/30 [==============================] - 2s 54ms/step - loss: 0.0165 - acc: 0.5495 - val_loss: 1.3308 - val_acc: 0.4528
Epoch 42/50
30/30 [==============================] - 2s 56ms/step - loss: 0.0162 - acc: 0.5694 - val_loss: 1.3274 - val_acc: 0.4642
Epoch 43/50
30/30 [==============================] - 2s 68ms/step - loss: 0.0319 - acc: 0.5529 - val_loss: 1.2909 - val_acc: 0.4525
Epoch 44/50
30/30 [==============================] - 2s 58ms/step - loss: 0.0163 - acc: 0.5564 - val_loss: 1.3119 - val_acc: 0.4631
Epoch 45/50
30/30 [==============================] - 1s 48ms/step - loss: 0.0162 - acc: 0.5667 - val_loss: 1.3168 - val_acc: 0.4604
Epoch 46/50
```

```
Epoch 46/50
30/30 [==============================] - 1s 50ms/step - loss: 0.0162 - acc: 0.5746 - val_loss: 1.3227 - val_acc: 0.4642
Epoch 47/50
30/30 [==============================] - 1s 49ms/step - loss: 0.0307 - acc: 0.5721 - val_loss: 1.3038 - val_acc: 0.4496
Epoch 48/50
30/30 [==============================] - 1s 46ms/step - loss: 0.0163 - acc: 0.5662 - val_loss: 1.3279 - val_acc: 0.4540
Epoch 49/50
30/30 [==============================] - 2s 56ms/step - loss: 0.0162 - acc: 0.5564 - val_loss: 1.3612 - val_acc: 0.4057
Epoch 50/50
30/30 [==============================] - 2s 62ms/step - loss: 0.0259 - acc: 0.5507 - val_loss: 1.3079 - val_acc: 0.4352
```

A42

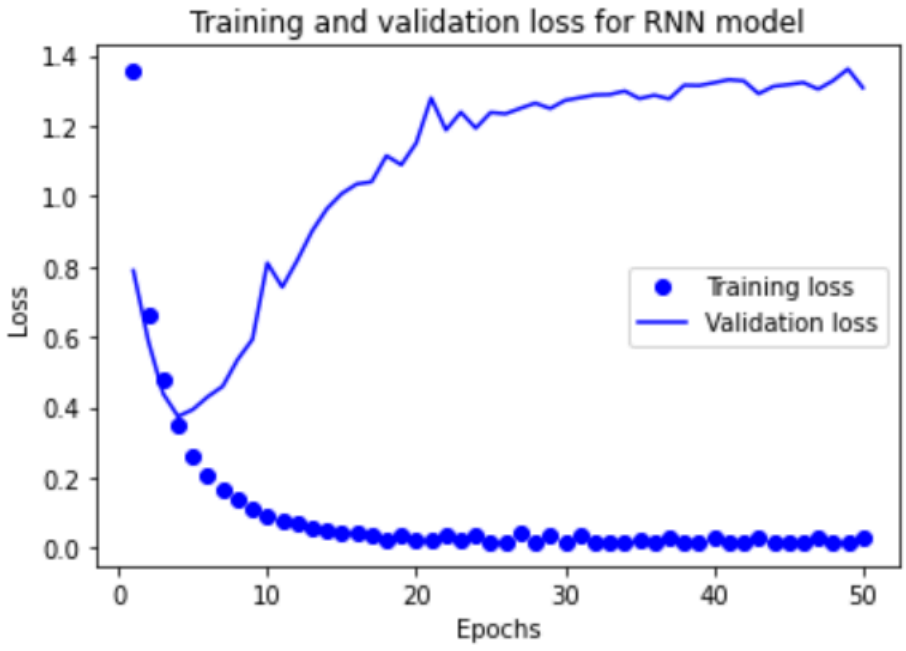
In [70]:

```
#Plot the Training and Validation loss for RNN Model
TS_RNN_model_hist_dict = TS_RNN_model_hist.history
TS_RNN_loss_values = TS_RNN_model_hist_dict['loss']
TS_RNN_val_loss_values = TS_RNN_model_hist_dict['val_loss']

TS_RNN_epochs = range(1, len(TS_RNN_model_hist_dict['loss']) + 1)

plt.plot(TS_RNN_epochs, TS_RNN_loss_values, 'bo', label='Training loss')
plt.plot(TS_RNN_epochs, TS_RNN_val_loss_values, 'b', label='Validation loss')
plt.title('Training and validation loss for RNN model')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

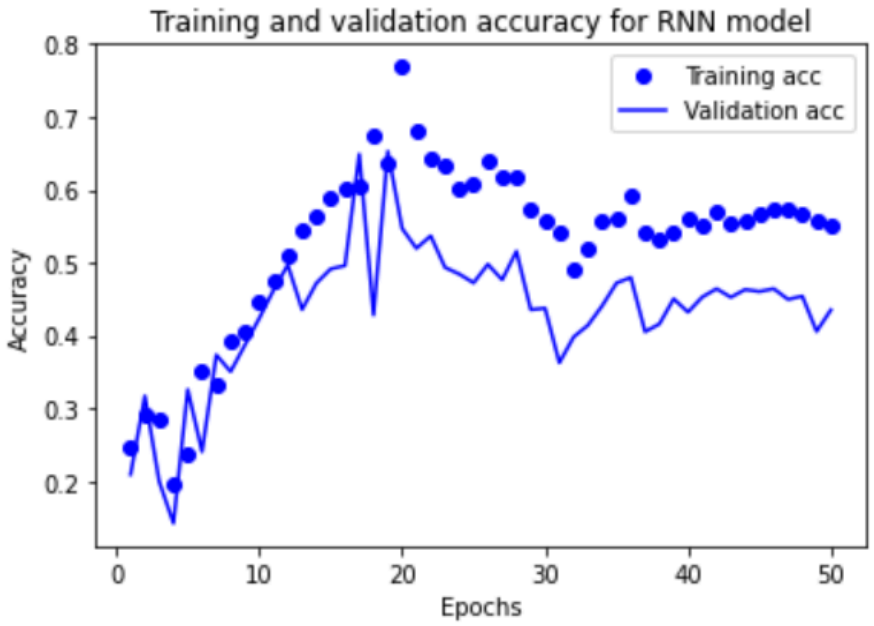


A43

In [71]:
```
#Plot the Training and Validation Accuracy for RNN Model
plt.clf()
TS_RNN_accuracy_values = TS_RNN_model_hist_dict['acc']
TS_RNN_val_accuracy_values = TS_RNN_model_hist_dict['val_acc']

plt.plot(TS_RNN_epochs, TS_RNN_accuracy_values, 'bo', label='Training acc')
plt.plot(TS_RNN_epochs, TS_RNN_val_accuracy_values, 'b', label='Validation acc')
plt.title('Training and validation accuracy for RNN model')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



A44

In [72]:
```
#Regularized model
TS_RNN_regularized_model = Sequential()
TS_RNN_regularized_model.add(layers.LSTM(32,
                     dropout=0.5,
                     recurrent_dropout=0.5,
                     input_shape=(100,)))
TS_RNN_regularized_model.add(layers.Dense(1))

TS_RNN_regularized_model.compile(optimizer=RMSprop(), loss='accuracy')
TS_RNN_regularized_model_hist = TS_RNN_regularized_model.fit(TS_x_train,TS_y_train,
                              steps_per_epoch=100,
                              epochs=50,
                              validation_data=(TS_x_val, TS_y_val))
```

```
Epoch 1/50
30/30 [==============================] - 16s 415ms/step - loss: 3.4467 - acc: 0.0489 - val_loss: 2.7633 - val_acc: 0.0000e
+00
Epoch 2/50
30/30 [==============================] - 2s 68ms/step - loss: 2.1017 - acc: 0.0380 - val_loss: 1.4798 - val_acc: 0.0000e+0
0
Epoch 3/50
30/30 [==============================] - 2s 67ms/step - loss: 1.4368 - acc: 7.3333e-04 - val_loss: 1.2520 - val_acc: 0.000
0e+00
Epoch 4/50
30/30 [==============================] - 2s 55ms/step - loss: 1.2141 - acc: 3.3333e-04 - val_loss: 1.1113 - val_acc: 0.000
0e+00
Epoch 5/50
30/30 [==============================] - 1s 50ms/step - loss: 1.0876 - acc: 1.3333e-04 - val_loss: 0.9729 - val_acc: 0.000
0e+00
Epoch 6/50
30/30 [==============================] - 1s 49ms/step - loss: 0.9864 - acc: 6.6667e-05 - val_loss: 0.8750 - val_acc: 0.000
0e+00
Epoch 7/50
```

```
Epoch 7/50
30/30 [==============================] - 1s 46ms/step - loss: 0.9142 - acc: 6.6667e-05 - val_loss: 0.7945 - val_acc: 0.000
0e+00
Epoch 8/50
30/30 [==============================] - 1s 50ms/step - loss: 0.8595 - acc: 6.6667e-05 - val_loss: 0.7695 - val_acc: 0.000
0e+00
Epoch 9/50
30/30 [==============================] - 2s 62ms/step - loss: 0.8216 - acc: 1.3333e-04 - val_loss: 0.7430 - val_acc: 0.000
0e+00
Epoch 10/50
30/30 [==============================] - 2s 56ms/step - loss: 0.7916 - acc: 0.0113 - val_loss: 0.7330 - val_acc: 0.0000e+0
0
Epoch 11/50
30/30 [==============================] - 2s 60ms/step - loss: 0.7654 - acc: 0.0155 - val_loss: 0.7118 - val_acc: 0.0000e+0
0
Epoch 12/50
30/30 [==============================] - 2s 62ms/step - loss: 0.7517 - acc: 0.0314 - val_loss: 0.7034 - val_acc: 0.0000e+0
0
Epoch 13/50
```

```
Epoch 13/50
30/30 [==============================] - 2s 55ms/step - loss: 0.7483 - acc: 0.0379 - val_loss: 0.6852 - val_acc: 0.0000e+0
0
Epoch 14/50
30/30 [==============================] - 2s 51ms/step - loss: 0.7305 - acc: 0.0327 - val_loss: 0.6858 - val_acc: 0.0000e+0
0
Epoch 15/50
30/30 [==============================] - 1s 49ms/step - loss: 0.7326 - acc: 0.0426 - val_loss: 0.6838 - val_acc: 0.0000e+0
0
Epoch 16/50
30/30 [==============================] - 1s 49ms/step - loss: 0.7164 - acc: 0.1096 - val_loss: 0.6796 - val_acc: 0.0000e+0
0
Epoch 17/50
30/30 [==============================] - 1s 45ms/step - loss: 0.7130 - acc: 0.1651 - val_loss: 0.6781 - val_acc: 0.0027
Epoch 18/50
30/30 [==============================] - 1s 48ms/step - loss: 0.7070 - acc: 0.2524 - val_loss: 0.6864 - val_acc: 0.0072
Epoch 19/50
30/30 [==============================] - 1s 47ms/step - loss: 0.7048 - acc: 0.3331 - val_loss: 0.6731 - val_acc: 0.4492
Epoch 20/50
```

```
Epoch 20/50
30/30 [==============================] - 1s 50ms/step - loss: 0.7005 - acc: 0.4819 - val_loss: 0.7177 - val_acc: 0.1843
Epoch 21/50
30/30 [==============================] - 2s 51ms/step - loss: 0.7125 - acc: 0.3280 - val_loss: 0.6799 - val_acc: 0.2801
Epoch 22/50
30/30 [==============================] - 2s 54ms/step - loss: 0.6837 - acc: 0.3214 - val_loss: 0.7027 - val_acc: 0.1972
Epoch 23/50
30/30 [==============================] - 1s 49ms/step - loss: 0.6895 - acc: 0.2972 - val_loss: 0.7016 - val_acc: 0.1609
Epoch 24/50
30/30 [==============================] - 2s 52ms/step - loss: 0.6598 - acc: 0.2785 - val_loss: 0.6663 - val_acc: 0.1904
Epoch 25/50
30/30 [==============================] - 2s 51ms/step - loss: 0.6574 - acc: 0.2703 - val_loss: 0.6945 - val_acc: 0.1517
Epoch 26/50
30/30 [==============================] - 2s 55ms/step - loss: 0.6547 - acc: 0.2531 - val_loss: 0.7285 - val_acc: 0.1417
Epoch 27/50
30/30 [==============================] - 2s 52ms/step - loss: 0.6394 - acc: 0.2444 - val_loss: 0.7477 - val_acc: 0.1116
Epoch 28/50
30/30 [==============================] - 2s 52ms/step - loss: 0.6158 - acc: 0.2297 - val_loss: 0.7546 - val_acc: 0.1205
Epoch 29/50
```

```
Epoch 29/50
30/30 [==============================] - 2s 57ms/step - loss: 0.6300 - acc: 0.2274 - val_loss: 0.7715 - val_acc: 0.1242
Epoch 30/50
30/30 [==============================] - 2s 52ms/step - loss: 0.6289 - acc: 0.2170 - val_loss: 0.7289 - val_acc: 0.1437
Epoch 31/50
30/30 [==============================] - 2s 51ms/step - loss: 0.6016 - acc: 0.2210 - val_loss: 0.8205 - val_acc: 0.0992
Epoch 32/50
30/30 [==============================] - 1s 50ms/step - loss: 0.6225 - acc: 0.2095 - val_loss: 0.8094 - val_acc: 0.1056
Epoch 33/50
30/30 [==============================] - 2s 54ms/step - loss: 0.6051 - acc: 0.2114 - val_loss: 0.8237 - val_acc: 0.1008
Epoch 34/50
30/30 [==============================] - 2s 58ms/step - loss: 0.6056 - acc: 0.2147 - val_loss: 0.8246 - val_acc: 0.0894
Epoch 35/50
30/30 [==============================] - 1s 45ms/step - loss: 0.6060 - acc: 0.2075 - val_loss: 0.8848 - val_acc: 0.1010
Epoch 36/50
30/30 [==============================] - 1s 47ms/step - loss: 0.5944 - acc: 0.2028 - val_loss: 0.9053 - val_acc: 0.0818
Epoch 37/50
30/30 [==============================] - 2s 52ms/step - loss: 0.5868 - acc: 0.2257 - val_loss: 0.8930 - val_acc: 0.0802
Epoch 38/50
```

```
Epoch 38/50
30/30 [==============================] - 2s 56ms/step - loss: 0.5675 - acc: 0.1978 - val_loss: 0.9044 - val_acc: 0.0990
Epoch 39/50
30/30 [==============================] - 2s 50ms/step - loss: 0.5639 - acc: 0.1952 - val_loss: 0.8994 - val_acc: 0.0759
Epoch 40/50
30/30 [==============================] - 2s 54ms/step - loss: 0.5522 - acc: 0.1915 - val_loss: 0.9464 - val_acc: 0.0617
Epoch 41/50
30/30 [==============================] - 2s 54ms/step - loss: 0.5704 - acc: 0.2677 - val_loss: 0.9375 - val_acc: 0.3975
Epoch 42/50
30/30 [==============================] - 2s 58ms/step - loss: 0.5608 - acc: 0.3739 - val_loss: 0.9449 - val_acc: 0.4070
Epoch 43/50
30/30 [==============================] - 2s 62ms/step - loss: 0.5447 - acc: 0.5082 - val_loss: 0.9486 - val_acc: 0.3991
Epoch 44/50
30/30 [==============================] - 2s 61ms/step - loss: 0.5519 - acc: 0.5308 - val_loss: 1.0051 - val_acc: 0.0736
Epoch 45/50
30/30 [==============================] - 1s 48ms/step - loss: 0.5573 - acc: 0.3087 - val_loss: 1.0124 - val_acc: 0.0744
Epoch 46/50
30/30 [==============================] - 2s 56ms/step - loss: 0.5629 - acc: 0.3030 - val_loss: 1.0160 - val_acc: 0.3971
Epoch 47/50
```

```
Epoch 47/50
30/30 [==============================] - 2s 54ms/step - loss: 0.5394 - acc: 0.3919 - val_loss: 1.0119 - val_acc: 0.3950
Epoch 48/50
30/30 [==============================] - 1s 48ms/step - loss: 0.5427 - acc: 0.5363 - val_loss: 0.9934 - val_acc: 0.4041
Epoch 49/50
30/30 [==============================] - 2s 56ms/step - loss: 0.5394 - acc: 0.3777 - val_loss: 1.0365 - val_acc: 0.4115
Epoch 50/50
30/30 [==============================] - 2s 52ms/step - loss: 0.5610 - acc: 0.5309 - val_loss: 1.0448 - val_acc: 0.4062
```

A45

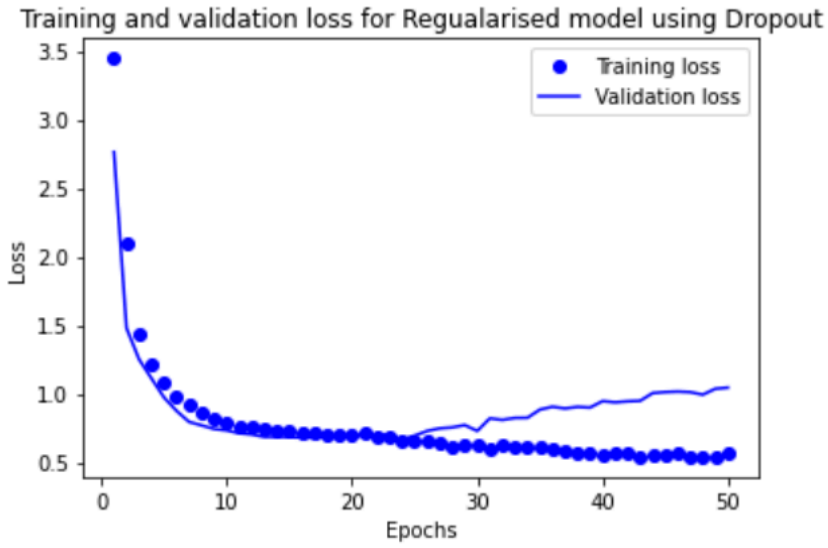
In [73]:

```
#Plot the Training and Validation loss for Regularized Model
TS_RNN_regularized_model_hist_dict = TS_RNN_regularized_model_hist.history
TS_RNN_regularized_loss_values = TS_RNN_regularized_model_hist_dict['loss']
TS_RNN_regularized_val_loss_values = TS_RNN_regularized_model_hist_dict['val_loss']

TS_RNN_regularized_epochs = range(1, len(TS_RNN_regularized_model_hist_dict['loss']) + 1)

plt.plot(TS_RNN_regularized_epochs, TS_RNN_regularized_loss_values, 'bo', label='Training loss')
plt.plot(TS_RNN_regularized_epochs, TS_RNN_regularized_val_loss_values, 'b', label='Validation loss')
plt.title('Training and validation loss for Regualarised model using Dropout')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

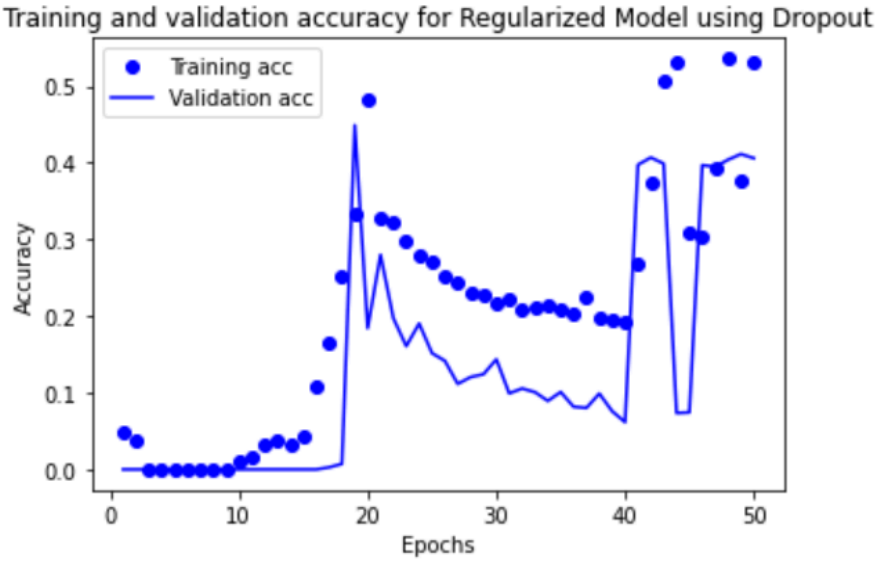


A46

In [74]:
```
#Plot the Training and Validation Accuracy for Regularized Model
plt.clf()
TS_RNN_regularized_accuracy_values = TS_RNN_regularized_model_hist_dict['acc']
TS_RNN_regularized_val_accuracy_values = TS_RNN_regularized_model_hist_dict['val_acc']

plt.plot(TS_RNN_regularized_epochs, TS_RNN_regularized_accuracy_values, 'bo', label='Training acc')
plt.plot(TS_RNN_regularized_epochs, TS_RNN_regularized_val_accuracy_values, 'b', label='Validation acc')
plt.title('Training and validation accuracy for Regularized Model using Dropout')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



A47

Stacked Recural Neural Networks(stacked RNNs)

In [75]:
```
#Built a stacked RNN model
Stacked_RNN_model = Sequential()
Stacked_RNN_model.add(layers.LSTM(32,
                     dropout=0.5,
                     recurrent_dropout=0.5,
                     return_sequences=True,
                     input_shape=(100,))
Stacked_RNN_model.add(layers.LSTM(65, activation='relu',
                     dropout=0.5,
                     recurrent_dropout=0.5))
Stacked_RNN_model.add(layers.Dense(1))

Stacked_RNN_model.compile(optimizer=RMSprop(), loss='accuracy')
Stacked_RNN_model_hist = Stacked_RNN_model.fit(TS_x_train,TS_y_train,
                              steps_per_epoch=100,
                              epochs=50,
                              validation_data=(TS_x_val, TS_y_val))
```

```
Epoch 1/50
30/30 [==============================] - 14s 347ms/step - loss: 4.4451 - acc: 0.2505 - val_loss: 2.8551 - val_acc: 0.4614
Epoch 2/50
30/30 [==============================] - 2s 73ms/step - loss: 2.9697 - acc: 0.2211 - val_loss: 1.9733 - val_acc: 0.3685
Epoch 3/50
30/30 [==============================] - 2s 62ms/step - loss: 1.8029 - acc: 0.1911 - val_loss: 1.5304 - val_acc: 0.5004
Epoch 4/50
30/30 [==============================] - 2s 58ms/step - loss: 1.4404 - acc: 0.1881 - val_loss: 1.2729 - val_acc: 0.5053
Epoch 5/50
30/30 [==============================] - 2s 53ms/step - loss: 1.2195 - acc: 0.1999 - val_loss: 1.0832 - val_acc: 0.5053
Epoch 6/50
30/30 [==============================] - 2s 51ms/step - loss: 1.0733 - acc: 0.2039 - val_loss: 0.9685 - val_acc: 0.5053
Epoch 7/50
30/30 [==============================] - 2s 53ms/step - loss: 0.9940 - acc: 0.2111 - val_loss: 0.8884 - val_acc: 0.5053
Epoch 8/50
30/30 [==============================] - 1s 49ms/step - loss: 0.9415 - acc: 0.2174 - val_loss: 0.8641 - val_acc: 0.5053
Epoch 9/50
30/30 [==============================] - 2s 52ms/step - loss: 0.9070 - acc: 0.2191 - val_loss: 0.8472 - val_acc: 0.5053
Epoch 10/50
```

```
Epoch 10/50
30/30 [==============================] - 2s 50ms/step - loss: 0.8740 - acc: 0.2099 - val_loss: 0.8169 - val_acc: 0.5053
Epoch 11/50
30/30 [==============================] - 2s 52ms/step - loss: 0.8461 - acc: 0.1751 - val_loss: 0.7895 - val_acc: 0.5050
Epoch 12/50
30/30 [==============================] - 2s 59ms/step - loss: 0.8207 - acc: 0.1543 - val_loss: 0.7737 - val_acc: 0.5030
Epoch 13/50
30/30 [==============================] - 2s 57ms/step - loss: 0.8013 - acc: 0.1602 - val_loss: 0.7434 - val_acc: 0.5013
Epoch 14/50
30/30 [==============================] - 1s 50ms/step - loss: 0.7841 - acc: 0.1506 - val_loss: 0.7238 - val_acc: 0.4549
Epoch 15/50
30/30 [==============================] - 1s 50ms/step - loss: 0.7763 - acc: 0.1455 - val_loss: 0.7141 - val_acc: 0.4650
Epoch 16/50
30/30 [==============================] - 2s 51ms/step - loss: 0.7678 - acc: 0.1507 - val_loss: 0.7109 - val_acc: 0.4643
Epoch 17/50
30/30 [==============================] - 1s 48ms/step - loss: 0.7689 - acc: 0.1660 - val_loss: 0.7078 - val_acc: 0.0385
Epoch 18/50
30/30 [==============================] - 1s 48ms/step - loss: 0.7654 - acc: 0.1762 - val_loss: 0.7038 - val_acc: 0.1461
Epoch 19/50
```

```
Epoch 19/50
30/30 [==============================] - 1s 50ms/step - loss: 0.7631 - acc: 0.1870 - val_loss: 0.7054 - val_acc: 0.2168
Epoch 20/50
30/30 [==============================] - 2s 59ms/step - loss: 0.7577 - acc: 0.1843 - val_loss: 0.6877 - val_acc: 0.0618
Epoch 21/50
30/30 [==============================] - 2s 59ms/step - loss: 0.7536 - acc: 0.1625 - val_loss: 0.6912 - val_acc: 0.2859
Epoch 22/50
30/30 [==============================] - 2s 52ms/step - loss: 0.7551 - acc: 0.1895 - val_loss: 0.6863 - val_acc: 0.1506
Epoch 23/50
30/30 [==============================] - 1s 46ms/step - loss: 0.7587 - acc: 0.1583 - val_loss: 0.6730 - val_acc: 0.0679
Epoch 24/50
30/30 [==============================] - 1s 32ms/step - loss: 0.7516 - acc: 0.1975 - val_loss: 0.6599 - val_acc: 0.0791
Epoch 25/50
30/30 [==============================] - 1s 31ms/step - loss: 0.7611 - acc: 0.1738 - val_loss: 0.6651 - val_acc: 0.3854
Epoch 26/50
30/30 [==============================] - 1s 27ms/step - loss: 0.7568 - acc: 0.2017 - val_loss: 0.6678 - val_acc: 0.4851
Epoch 27/50
30/30 [==============================] - 1s 27ms/step - loss: 0.7524 - acc: 0.2223 - val_loss: 0.6504 - val_acc: 0.2499
Epoch 28/50
```

```
Epoch 28/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7597 - acc: 0.2263 - val_loss: 0.6342 - val_acc: 0.4493
Epoch 29/50
30/30 [==============================] - 1s 23ms/step - loss: 0.7596 - acc: 0.1442 - val_loss: 0.6287 - val_acc: 0.5111
Epoch 30/50
30/30 [==============================] - 1s 22ms/step - loss: 0.7592 - acc: 0.1282 - val_loss: 0.6520 - val_acc: 0.0032
Epoch 31/50
30/30 [==============================] - 1s 24ms/step - loss: 0.7579 - acc: 0.1397 - val_loss: 0.6374 - val_acc: 0.0091
Epoch 32/50
30/30 [==============================] - 1s 24ms/step - loss: 0.7784 - acc: 0.1509 - val_loss: 0.6144 - val_acc: 0.0017
Epoch 33/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7551 - acc: 0.0867 - val_loss: 0.6270 - val_acc: 0.0000e+0
0
Epoch 34/50
30/30 [==============================] - 1s 25ms/step - loss: 0.7701 - acc: 0.1748 - val_loss: 0.6402 - val_acc: 0.1475
Epoch 35/50
30/30 [==============================] - 1s 27ms/step - loss: 0.7785 - acc: 0.1727 - val_loss: 0.6551 - val_acc: 0.0020
Epoch 36/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7806 - acc: 0.1040 - val_loss: 0.6418 - val_acc: 0.1886
```

```
Epoch 37/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7761 - acc: 0.1651 - val_loss: 0.6339 - val_acc: 0.0000e+0
0
Epoch 38/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7754 - acc: 0.2457 - val_loss: 0.6257 - val_acc: 0.3751
Epoch 39/50
30/30 [==============================] - 1s 25ms/step - loss: 0.7822 - acc: 0.2991 - val_loss: 0.6444 - val_acc: 0.2244
Epoch 40/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7830 - acc: 0.2459 - val_loss: 0.6677 - val_acc: 0.0000e+0
0
Epoch 41/50
30/30 [==============================] - 1s 31ms/step - loss: 0.7732 - acc: 0.0771 - val_loss: 0.6570 - val_acc: 0.0060
Epoch 42/50
30/30 [==============================] - 1s 28ms/step - loss: 0.7847 - acc: 0.2390 - val_loss: 0.6526 - val_acc: 0.1996
Epoch 43/50
30/30 [==============================] - 1s 26ms/step - loss: 0.7846 - acc: 0.2947 - val_loss: 0.6511 - val_acc: 0.4000
Epoch 44/50
30/30 [==============================] - 1s 28ms/step - loss: 0.7919 - acc: 0.3789 - val_loss: 0.6271 - val_acc: 0.4801
Epoch 45/50
```

```
Epoch 45/50
30/30 [==============================] - 1s 33ms/step - loss: 0.7944 - acc: 0.4153 - val_loss: 0.6460 - val_acc: 0.4604
Epoch 46/50
30/30 [==============================] - 1s 29ms/step - loss: 0.7802 - acc: 0.4026 - val_loss: 0.6597 - val_acc: 0.4868
Epoch 47/50
30/30 [==============================] - 1s 27ms/step - loss: 0.7775 - acc: 0.2949 - val_loss: 0.6562 - val_acc: 0.0000e+0
0
Epoch 48/50
30/30 [==============================] - 1s 26ms/step - loss: 0.8001 - acc: 0.0807 - val_loss: 0.6673 - val_acc: 0.0000e+0
0
Epoch 49/50
30/30 [==============================] - 1s 34ms/step - loss: 0.7957 - acc: 0.1284 - val_loss: 0.6685 - val_acc: 0.0000e+0
0
Epoch 50/50
30/30 [==============================] - 1s 30ms/step - loss: 0.7954 - acc: 0.4215 - val_loss: 0.6854 - val_acc: 0.4887
```

A48

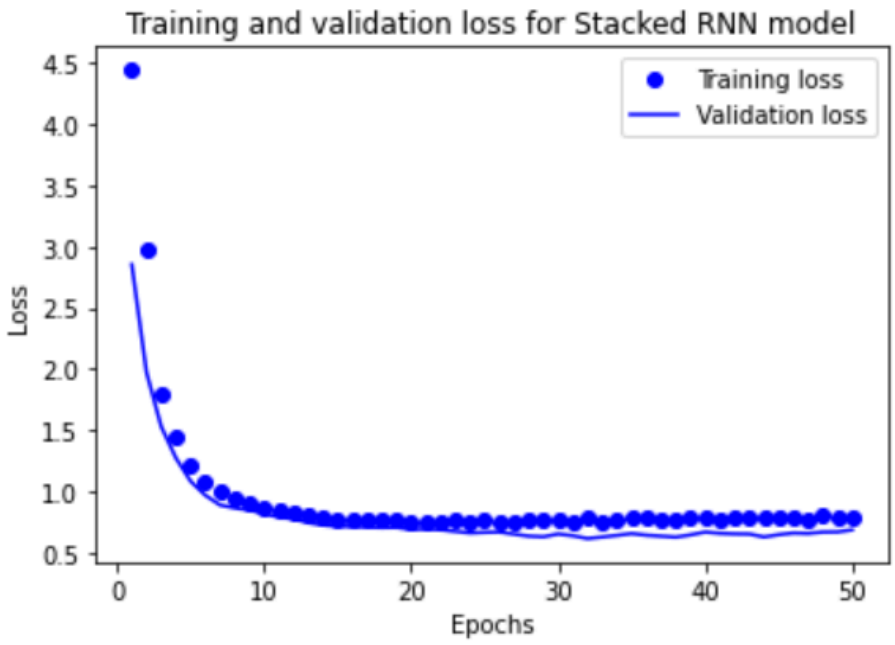
In [76]:

```
#Plot the Training and Validation loss for Stacked RNN model
Stacked_RNN_model_hist_dict = Stacked_RNN_model_hist.history
Stacked_RNN_loss_values = Stacked_RNN_model_hist_dict['loss']
Stacked_RNN_val_loss_values = Stacked_RNN_model_hist_dict['val_loss']

Stacked_RNN_epochs = range(1, len(Stacked_RNN_model_hist_dict['loss']) + 1)

plt.plot(Stacked_RNN_epochs, Stacked_RNN_loss_values, 'bo', label='Training loss')
plt.plot(Stacked_RNN_epochs, Stacked_RNN_val_loss_values, 'b', label='Validation loss')
plt.title('Training and validation loss for Stacked RNN model')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



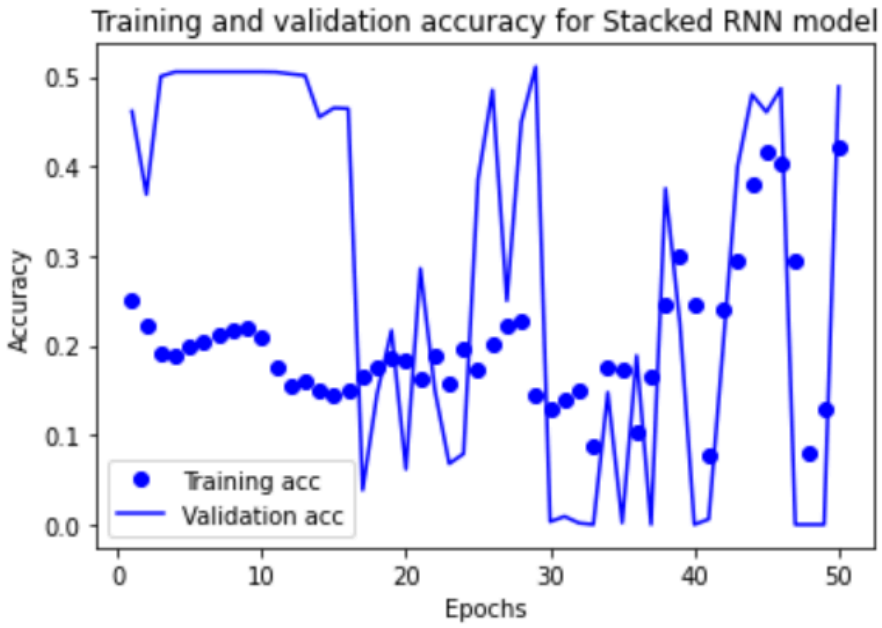
A49

In [77]:

```
#Plot the Training and Validation Accuracy for Stacked RNN model
plt.clf()
Stacked_RNN_accuracy_values = Stacked_RNN_model_hist_dict['acc']
Stacked_RNN_val_accuracy_values = Stacked_RNN_model_hist_dict['val_acc']

plt.plot(Stacked_RNN_epochs, Stacked_RNN_accuracy_values, 'bo', label='Training acc')
plt.plot(Stacked_RNN_epochs, Stacked_RNN_val_accuracy_values, 'b', label='Validation acc')
plt.title('Training and validation accuracy for Stacked RNN model')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



A50

In [78]:
```
#Build the bis-LSTM model
bi_lstm_model = Sequential()
bi_lstm_model.add(layers.Embedding(100, 32))
bi_lstm_model.add(layers.Bidirectional(layers.LSTM(32)))
bi_lstm_model.add(layers.Dense(1, activation='sigmoid'))

bi_lstm_model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['acc'])
bi_lstm_model_hist = bi_lstm_model.fit(TS_x_train,TS_y_train,
                                steps_per_epoch=100,
                                epochs=50,
                                validation_data=(TS_x_val, TS_y_val))
```

```
Epoch 1/50
49/49 [==============================] - 7s 152ms/step - loss: 0.8112 - acc: 0.4130 - val_loss: 0.6526 - val_acc: 0.4774
Epoch 2/50
49/49 [==============================] - 2s 49ms/step - loss: 0.8057 - acc: 0.4680 - val_loss: 0.6610 - val_acc: 0.4752
Epoch 3/50
49/49 [==============================] - 2s 34ms/step - loss: 0.7992 - acc: 0.4598 - val_loss: 0.6645 - val_acc: 0.6279
Epoch 4/50
49/49 [==============================] - 2s 33ms/step - loss: 0.8080 - acc: 0.3621 - val_loss: 0.6673 - val_acc: 0.0000e+0
0
Epoch 5/50
49/49 [==============================] - 2s 33ms/step - loss: 0.7954 - acc: 0.1112 - val_loss: 0.6759 - val_acc: 0.0000e+0
0
Epoch 6/50
49/49 [==============================] - 1s 27ms/step - loss: 0.8131 - acc: 0.0649 - val_loss: 0.6571 - val_acc: 0.0000e+0
0
Epoch 7/50
49/49 [==============================] - 1s 26ms/step - loss: 0.8251 - acc: 0.2982 - val_loss: 0.6782 - val_acc: 0.4928
Epoch 8/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8253 - acc: 0.2902 - val_loss: 0.6780 - val_acc: 0.5018
```

```
Epoch 8/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8253 - acc: 0.2902 - val_loss: 0.6780 - val_acc: 0.5018
Epoch 9/50
49/49 [==============================] - 1s 23ms/step - loss: 0.8067 - acc: 0.1738 - val_loss: 0.6908 - val_acc: 0.3589
Epoch 10/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8044 - acc: 0.3605 - val_loss: 0.6639 - val_acc: 0.4738
Epoch 11/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8148 - acc: 0.4409 - val_loss: 0.6647 - val_acc: 0.4760
Epoch 12/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8116 - acc: 0.2760 - val_loss: 0.6625 - val_acc: 0.2577
Epoch 13/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8036 - acc: 0.3112 - val_loss: 0.6574 - val_acc: 0.2460
Epoch 14/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8131 - acc: 0.2280 - val_loss: 0.6671 - val_acc: 0.0015
Epoch 15/50
49/49 [==============================] - 1s 27ms/step - loss: 0.8090 - acc: 0.1935 - val_loss: 0.6534 - val_acc: 0.0000e+0
0
Epoch 16/50
49/49 [==============================] - 2s 33ms/step - loss: 0.8080 - acc: 0.3265 - val_loss: 0.6516 - val_acc: 0.4916
```

```
Epoch 17/50
49/49 [==============================] - 1s 29ms/step - loss: 0.8199 - acc: 0.3602 - val_loss: 0.6516 - val_acc: 0.4911
Epoch 18/50
49/49 [==============================] - 1s 23ms/step - loss: 0.8158 - acc: 0.2816 - val_loss: 0.6519 - val_acc: 0.0027
Epoch 19/50
49/49 [==============================] - 1s 23ms/step - loss: 0.7990 - acc: 0.0878 - val_loss: 0.6496 - val_acc: 0.0577
Epoch 20/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8007 - acc: 0.0803 - val_loss: 0.6580 - val_acc: 0.0000e+0
0
Epoch 21/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8150 - acc: 0.0734 - val_loss: 0.6545 - val_acc: 0.1604
Epoch 22/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8099 - acc: 0.0585 - val_loss: 0.6567 - val_acc: 0.1411
Epoch 23/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8235 - acc: 0.0827 - val_loss: 0.6814 - val_acc: 0.0000e+0
0
Epoch 24/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8087 - acc: 0.0207 - val_loss: 0.6597 - val_acc: 0.0000e+0
0
```

```
Epoch 25/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8092 - acc: 0.3068 - val_loss: 0.6622 - val_acc: 0.2214
Epoch 26/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8122 - acc: 0.3190 - val_loss: 0.6815 - val_acc: 0.4947
Epoch 27/50
49/49 [==============================] - 1s 26ms/step - loss: 0.8041 - acc: 0.3684 - val_loss: 0.6729 - val_acc: 0.2810
Epoch 28/50
49/49 [==============================] - 1s 27ms/step - loss: 0.8149 - acc: 0.2684 - val_loss: 0.6546 - val_acc: 0.2819
Epoch 29/50
49/49 [==============================] - 1s 26ms/step - loss: 0.8063 - acc: 0.2688 - val_loss: 0.6616 - val_acc: 0.4947
Epoch 30/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8053 - acc: 0.3948 - val_loss: 0.6449 - val_acc: 0.4947
Epoch 31/50
49/49 [==============================] - 1s 23ms/step - loss: 0.8218 - acc: 0.1296 - val_loss: 0.6363 - val_acc: 0.0000e+0
0
Epoch 32/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8192 - acc: 0.0254 - val_loss: 0.6629 - val_acc: 0.0000e+0
0
```

```
0
Epoch 33/50
49/49 [==============================] - 1s 29ms/step - loss: 0.8085 - acc: 0.0297 - val_loss: 0.6629 - val_acc: 0.0000e+0
0
Epoch 34/50
49/49 [==============================] - 1s 28ms/step - loss: 0.8055 - acc: 0.0712 - val_loss: 0.6597 - val_acc: 0.0132
Epoch 35/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8161 - acc: 0.0680 - val_loss: 0.6563 - val_acc: 0.1955
Epoch 36/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8153 - acc: 0.3451 - val_loss: 0.6597 - val_acc: 0.2453
Epoch 37/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8197 - acc: 0.1508 - val_loss: 0.6709 - val_acc: 0.1456
Epoch 38/50
49/49 [==============================] - 1s 25ms/step - loss: 0.8179 - acc: 0.0824 - val_loss: 0.6637 - val_acc: 0.0835
Epoch 39/50
49/49 [==============================] - 1s 27ms/step - loss: 0.8238 - acc: 0.1184 - val_loss: 0.6594 - val_acc: 0.2339
Epoch 40/50
49/49 [==============================] - 1s 29ms/step - loss: 0.8084 - acc: 0.3114 - val_loss: 0.6386 - val_acc: 0.7708
Epoch 41/50
49/49 [==============================] - 1s 27ms/step - loss: 0.8139 - acc: 0.4465 - val_loss: 0.6596 - val_acc: 0.4895
```

```
Epoch 42/50
49/49 [==============================] - 1s 28ms/step - loss: 0.8153 - acc: 0.2786 - val_loss: 0.6727 - val_acc: 0.1508
Epoch 43/50
49/49 [==============================] - 2s 33ms/step - loss: 0.8048 - acc: 0.0786 - val_loss: 0.6535 - val_acc: 0.0000e+0
0
Epoch 44/50
49/49 [==============================] - 2s 31ms/step - loss: 0.8113 - acc: 0.0525 - val_loss: 0.6388 - val_acc: 0.0000e+0
0
Epoch 45/50
49/49 [==============================] - 1s 23ms/step - loss: 0.8174 - acc: 0.0671 - val_loss: 0.6523 - val_acc: 0.0000e+0
0
Epoch 46/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8131 - acc: 0.1216 - val_loss: 0.6551 - val_acc: 0.1584
Epoch 47/50
49/49 [==============================] - 1s 24ms/step - loss: 0.8136 - acc: 0.2588 - val_loss: 0.6744 - val_acc: 0.0024
Epoch 48/50
49/49 [==============================] - 1s 23ms/step - loss: 0.8152 - acc: 0.1503 - val_loss: 0.6612 - val_acc: 0.2283
Epoch 49/50
49/49 [==============================] - 1s 27ms/step - loss: 0.8033 - acc: 0.2425 - val_loss: 0.6780 - val_acc: 0.5759
```

```
Epoch 50/50
49/49 [==============================] - 2s 33ms/step - loss: 0.8112 - acc: 0.1638 - val_loss: 0.6615 - val_acc: 0.1720
```

A51

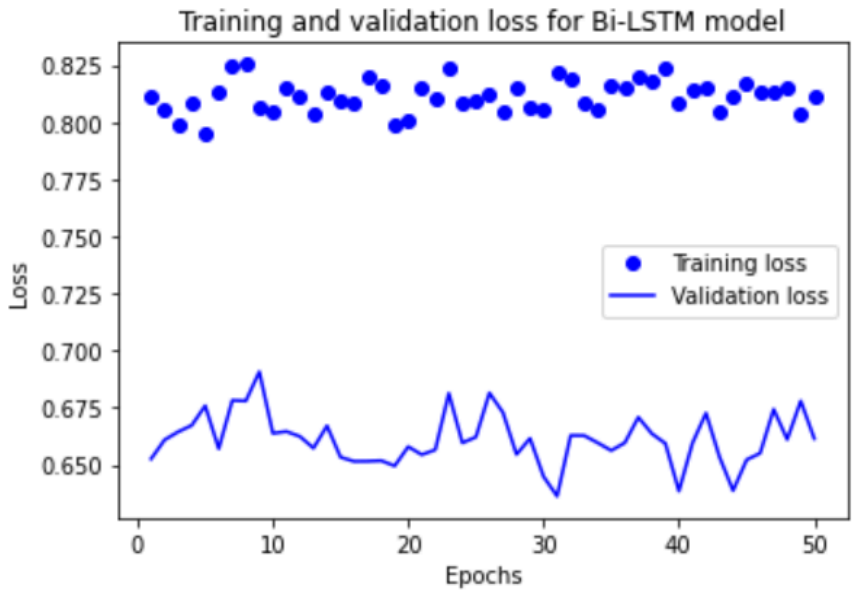
In [79]:

```
#Plot the Training and Validation loss for Bi-LSTM model
bi_lstm_model_hist_dict = bi_lstm_model_hist.history
bi_lstm_loss_values = bi_lstm_model_hist_dict['loss']
bi_lstm_val_loss_values = bi_lstm_model_hist_dict['val_loss']

bi_lstm_epochs = range(1, len(bi_lstm_model_hist_dict['loss']) + 1)

plt.plot(bi_lstm_epochs, bi_lstm_loss_values, 'bo', label='Training loss')
plt.plot(bi_lstm_epochs, bi_lstm_val_loss_values, 'b', label='Validation loss')
plt.title('Training and validation loss for Bi-LSTM model')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

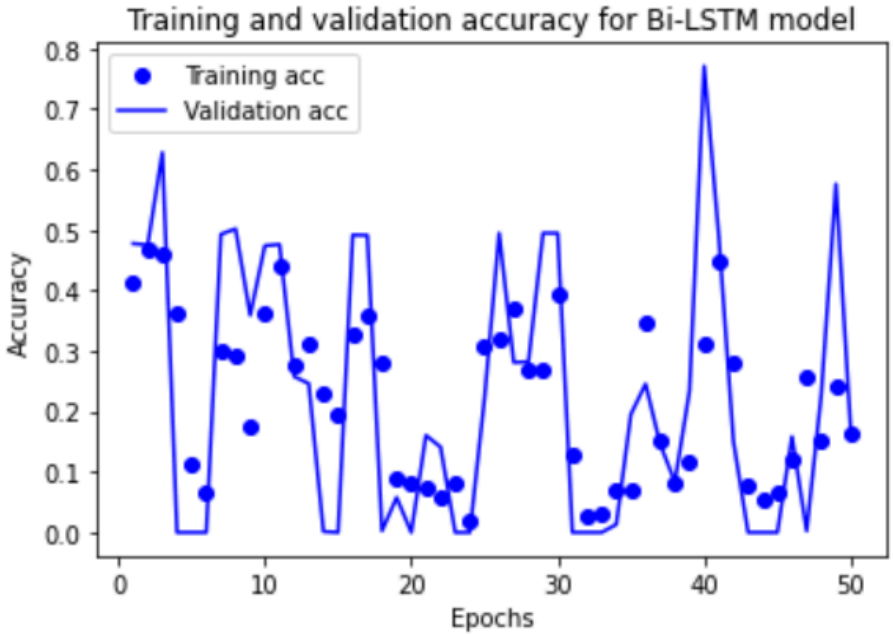


A52

In [80]:
```
#Plot the Training and Validation Accuracy for Bi-LSTM model
plt.clf()
bi_lstm_accuracy_values = bi_lstm_model_hist_dict['acc']
bi_lstm_val_accuracy_values = bi_lstm_model_hist_dict['val_acc']

plt.plot(bi_lstm_epochs, bi_lstm_accuracy_values, 'bo', label='Training acc')
plt.plot(bi_lstm_epochs, bi_lstm_val_accuracy_values, 'b', label='Validation acc')
plt.title('Training and validation accuracy for Bi-LSTM model')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```



A53

Testing

In [81]:
```
#Test using the test set
Predict_Y = bi_lstm_model.predict(TS_x_test)
print(Predict_Y)
```

```
71/71 [==============================] - 141s 2s/step
[[0.5045497 ]
 [0.5045973 ]
 [0.50486976]
 ...
 [0.50800997]
 [0.5048558 ]
 [0.5047469 ]]
```

A54

In [82]:
```
#F1 Score for Bi-LSTM Model
Predicted_bi_lstm_Y = bi_lstm_model.predict(TS_x_test)
F1_score_bi_lstm = bi_lstm_model.score(TS_y_test,Predicted_bi_lstm_Y)
print(F1_score_bi_lstm)
```

```
0.29
```

A55

In [85]:
```
#roc_auc socre for Bi-LSTM model
Roc_auc_score_bi_lstm = roc_auc_score(TS_y_test, Predicted_bis_lstm_Y, average=None)
print(Roc_auc_score_bi_lstm)
```

```
0.7
```

A56

Evaluation

In [86]:
```
#RNN model
RNN_model = Sequential()
RNN_model.add(layers.GRU(32, input_shape=(100,))
RNN_model.add(layers.Dense(1))

RNN_model.compile(optimizer=RMSprop(), loss='mae')
RNN_history = RNN_model.fit(TS_x_train,TS_y_train,
                            steps_per_epoch=100,
                            epochs=50,
                            validation_data=(TS_x_val, TS_y_val))
```

In [87]:
```
#F1 Score for RNN Model
Predicted_RNN_Y = RNN_model.predict(TS_x_test)
F1_score_RNN = RNN_model.score(TS_y_test,Predicted_RNN_Y)
print(F1_score_RNN)
```

```
0.28
```

In [88]:
```
#Roc_auc score for RNN model
Roc_auc_score_RNN = roc_auc_score(TS_y_test, Predicted_RNN_Y, average=None)
print(Roc_auc_score_RNN)
```

```
0.72
```

A57

In [87]:
```
#CNN model
CNN_model = models.Sequential()
CNN_model.add(layers.Conv2D(32, (3, 3), activation='relu',
                        input_shape=(100,)))
CNN_model.add(layers.MaxPooling2D((2, 2)))
CNN_model.add(layers.Conv2D(65, (3, 3), activation='relu'))
CNN_model.add(layers.MaxPooling2D((2, 2)))
CNN_model.add(layers.Conv2D(120, (3, 3), activation='relu'))
CNN_model.add(layers.MaxPooling2D((2, 2)))
CNN_model.add(layers.Conv2D(120, (3, 3), activation='relu'))
CNN_model.add(layers.MaxPooling2D((2, 2)))
CNN_model.add(layers.Flatten())
CNN_model.add(layers.Dense(512, activation='relu'))
CNN_model.add(layers.Dense(1, activation='sigmoid'))
CNN_model.compile(loss='categorical_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-4),
              metrics=['acc'])
CNN_history = CNN_model.fit(TS_x_train,TS_y_train,
                      steps_per_epoch=100,
                      epochs=50,
                      validation_data=(TS_x_val, TS_y_val))
```

In [90]:
```
#F1 score for CNN model
Predicted_CNN_Y = CNN_model.predict(TS_x_test)
F1_score_CNN = CNN_model.score(TS_y_test,Predicted_CNN_Y)
print(F1_score_CNN)
```

```
0.24
```

In [91]:
```
#Roc_auc score for CNN model
Roc_auc_score_CNN = roc_auc_score(TS_y_test, Predicted_CNN_Y, average=None)
print(Roc_auc_score_CNN)
```

```
0.5
```

CHAPTER 8:REFERENCES

1. *TEAM, W. (2022) World Mental Health Report: Transforming mental health for all, World Health Organization. Available at: https://www.who.int/publications/i/item/9789240049338 (Accessed: 08 January 2024).*
2. *Edwards, E. and Jackson, H. (2023) Social media is driving teen mental health crisis, Surgeon General warns, NBCNews.com. Available at: https://www.nbcnews.com/health/health-news/social-media-mental-health-anxiety-depression-teens-surgeon-general-rcna85575 (Accessed: 08 January 2024).*
3. *Coe, E. et al. (2023) Gen Z mental health: The impact of tech and Social Media, McKinsey & Company. Available at: https://www.mckinsey.com/mhi/our-insights/gen-z-mental-health-the-impact-of-tech-and-social-media (Accessed: 08 January 2024).*
4. *The social dilemma: Social media and your mental health (2023) Here's How Social Media Affects Your Mental Health | McLean Hospital. Available at: https://www.mcleanhospital.org/essential/it-or-not-social-medias-affecting-your-mental-health (Accessed: 08 January 2024).*
5. *Kour, H. et al. (2022) An hybrid deep learning approach for depression prediction from user tweets using feature-rich CNN and bi-directional LSTM, Multimedia Tools and Applications. Available at: https://dl.acm.org/doi/abs/10.1007/s11042-022-12648-y (Accessed: 08 January 2024).*
6. *Angskun, J., Tipprasert, S. and Angskun, T. (2022) Big Data Analytics on social networks for real-time Depression Detection - Journal of Big Data, SpringerOpen. Available at: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00622-2 (Accessed: 08 January 2024).*

7. *Chiu, C.Y. et al. (2020) Multimodal depression detection on Instagram considering time interval of posts - journal of intelligent information systems, SpringerLink. Available at: https://link.springer.com/article/10.1007/s10844-020-00599-5 (Accessed: 08 January 2024).*
8. *Nijhawan, T., Attigeri, G. and Ananthakrishna, T. (2022) Stress detection using natural language processing and machine learning over social interactions - Journal of Big Data, SpringerOpen. Available at: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00575-6 (Accessed: 08 January 2024).*
9. *Prakash, P. (2023) Understanding baseline models in machine learning, Medium. Available at:https://medium.com/@preethiprakash29/understanding-baseline-models-in-machine-learning-3ed94f03d645 (Accessed: 08 January 2024).*
10. *souvikahmed071 (2023) Correlation between SM and mental health, Kaggle. Available at: https://www.kaggle.com/code/souvikahmed071/correlation-between-sm-and-mental-health (Accessed: 08 January 2024).*
11. *Anwar, S. (2023) Mental health trends in the age of Social Media, Kaggle. Available at: https://www.kaggle.com/code/syedanwarafridi/mental-health-trends-in-the-age-of-social-media (Accessed: 08 January 2024).*
12. *Navlani, A. (2023) Python decision tree classification tutorial: Scikit-Learn Decisiontreeclassifier, DataCamp. Available at: https://www.datacamp.com/tutorial/decision-tree-classification-python (Accessed: 08 January 2024).*
13. *Simplilearn (2023) Random Forest algorithm, Simplilearn.com. Available at: https://www.simplilearn.com/tutorials/machine-learning-tutorial/random-forest-algorithm (Accessed: 08 January 2024).*
14. *Horev, R. (2018) Bert explained: State of the art language model for NLP, Medium. Available at:*

https://towardsdatascience.com/bert-explained-state-of-the-art-language-model-for-nlp-f8b21a9b6270 (Accessed: 08 January 2024).

15. *Biswal, A. (2023) Power of recurrent neural networks (RNN): Revolutionizing ai, Simplilearn.com. Available at: https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn (Accessed: 08 January 2024).*
16. *Keita, Z. (2023) An introduction to Convolutional Neural Networks: A comprehensive guide to CNNS in Deep learning, DataCamp. Available at: https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns (Accessed: 08 January 2024).*
17. *Thevapalan, A. (2023) The Machine Learning Workflow explained (and how you can practice it now), Medium. Available at: https://towardsdatascience.com/the-machine-learning-workflow-explained-557abf882079 (Accessed: 11 March 2024).*
18. *Chollet, F. (2019) Deep Learning with python, · Deep Learning with Python. Available at: https://livebook.manning.com/book/deep-learning-with-python/chapter-6/202 (Accessed: 11 March 2024).*
19. *Zvornicanin, W. by: E. (2023) Differences between bidirectional and Unidirectional LSTM, Baeldung on Computer Science. Available at: https://www.baeldung.com/cs/bidirectional-vs-unidirectional-lstm (Accessed: 11 March 2024).*
20. *Nabil, M. (2023) Unveiling the diversity: A comprehensive guide to types of CNN Architectures, Medium. Available at: https://medium.com/@navarai/unveiling-the-diversity-a-comprehensive-guide-to-types-of-cnn-architectures-9d70da0b4521 (Accessed: 11 March 2024).*
21. *GfG (2022) How to convert Numpy Array to tensor?, GeeksforGeeks. Available at: https://www.geeksforgeeks.org/how-to-convert-numpy-array-to-tensor/ (Accessed: 11 March 2024).*
22. *Verma, J. (2023) How to normalize data using scikit-learn in Python, DigitalOcean. Available at:*

https://www.digitalocean.com/community/tutorials/normalize-data-in-python (Accessed: 11 March 2024).

23. *Codebasics (2021) Simple explanation of LSTM | deep learning tutorial 36 (Tensorflow, Keras & Python), YouTube. Available at: https://www.youtube.com/watch?v=LfnrRPFhkuY (Accessed: 11 March 2024).*
24.